TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

KATHMANDU ENGINEERING COLLEGE

KALIMATI, KATHMANDU

Minor Project Report

On

NETWORK INTRUSION DETECTION SYSTEM (NIDS)

[Code Number: ENCT 354]

BY

Biraj Kandel	[KAT080BCT027]
Grisha Shrestha	[KAT080BCT035]
Hardik Thapaliya	[KAT080BCT036]
Mohit Bhatta	[KAT080BCT045]

TO

DEPARTMENT OF COMPUTER ENGINEERING

KATHMANDU, NEPAL

Bhadra, 2083

TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

Kathmandu Engineering College

Department of Computer Engineering

NETWORK INTRUSION DETECTION SYSTEM (NIDS)

[Code Number: ENCT 354]

MINOR PROJECT REPORT SUBMITTED TO

THE DEPARTMENT OF COMPUTER ENGINEERING

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR

THE BACHELOR OF ENGINEERING IN COMPUTER ENGINEERING

BY

Biraj Kandel	[KAT080BCT027]
Grisha Shrestha	[KAT080BCT035]
Hardik Thapaliya	[KAT080BCT036]
Mohit Bhatta	[KAT080BCT045]

Kathmandu, Nepal

Bhadra, 2083

Acknowledgement

We would like to express our sincere gratitude to Department of Computer Engineering, Kathmandu Engineering College, for providing us the opportunity to undertake this Minor Project on *Network Intrusion Detection System*.

We are especially grateful to Lecturer Er. Krista Byanju, our project coordinator, for her continuous guidance, support, and for providing essential information throughout the project. We would also like to thank Associate Professor Er. Sharad Chandra Joshi and Senior Lecturer Er. Ritu Bajracharya for their valuable guidance throughout the course of this project.
We also extend our appreciation to Associate Professor Er. Sudeep Shakya, Head of the Department of Computer Engineering, Associate Professor Er. Kunjan Amatya, Deputy Head of the Department and all the faculty members of the department for providing the necessary resources, facilities, and academic environment that enabled us to carry out this project successfully.

We also thank the Canadian Institute for Cybersecurity, University of New Brunswick, for making the CSE-CIC-IDS2018 dataset publicly available for research and educational purposes.

Finally, we are grateful to our families and friends for their patience and motivation during the course of this project.

Abstract

The rapid growth of computer networks has resulted in a significant increase in the volume and diversity of network attacks, making manual monitoring of network traffic impractical and unreliable. This project presents a machine learning and deep learning based Network Intrusion Detection System (NIDS) trained and evaluated using the CSE-CIC-IDS2018 dataset, which contains normal traffic and fourteen different attack types, including denial-of-service, brute-force, botnet, web attacks, and infiltration. The raw traffic records were cleaned, de-duplicated, and reduced to the 30 most informative flow features using mutual-information-based feature selection. The severe class imbalance was addressed by limiting the dominant benign class and synthetically augmenting underrepresented attack classes using CTGAN. Four classifiers—HistGradientBoosting, XGBoost, Random Forest, and Multi-Layer Perceptron (MLP)—were optimized using the Optuna hyperparameter optimization framework and evaluated on a held-out test set. HistGradientBoosting achieved the best performance, with 98.03% accuracy and a macro F1-score of 0.8627, and was selected as the default detection model, while the MLP provided a deep learning baseline. The trained model was integrated into a Django-based web application supporting single-flow classification, batch dataset analysis, and live network traffic detection. The live detection pipeline captures packets, constructs network flows, extracts the required features, applies the same preprocessing used during training, and generates predictions in real time. A controlled brute-force attack was successfully captured and classified as an attack, validating the complete end-to-end workflow and demonstrating the effectiveness of the proposed system for automated and scalable network intrusion detection.

Keywords: Network Intrusion Detection, Machine Learning, CSE-CIC-IDS2018, Traffic Classification, Feature Selection, Class Imbalance, Live Detection.

Contents

List of Figures

List of Tables

List of Abbreviations

API	Application Programming Interface
CIC	Canadian Institute for Cybersecurity
CSV	Comma-Separated Values
CTGAN	Conditional Tabular Generative Adversarial Network
DDoS	Distributed Denial of Service
DoS	Denial of Service
EDA	Exploratory Data Analysis
F1	F1-Score (harmonic mean of precision and recall)
FTP	File Transfer Protocol
HGB	Histogram-based Gradient Boosting
HOIC	High Orbit Ion Cannon
HTTP	Hypertext Transfer Protocol
IAT	Inter-Arrival Time
IDS	Intrusion Detection System
IP	Internet Protocol
JSON	JavaScript Object Notation
LOIC	Low Orbit Ion Cannon
MI	Mutual Information
ML	Machine Learning
MLP	Multi-Layer Perceptron
NIDS	Network Intrusion Detection System
RF	Random Forest
SSH	Secure Shell
TCP	Transmission Control Protocol
TPE	Tree-structured Parzen Estimator
UDP	User Datagram Protocol
UML	Unified Modeling Language
XGBoost	Extreme Gradient Boosting
XSS	Cross-Site Scripting

1. Introduction

1.1 Background Theory

A computer network is a collection of interconnected computing devices that exchange data through a shared communication medium governed by a common set of protocols. Modern networks carry a very large volume of traffic in the form of *packets*, which are grouped into *flows*: sequences of packets that share the same source and destination addresses, ports and transport protocol. The behaviour of a flow—its duration, the number and size of packets it carries, the timing between packets, and the transport-layer flags it sets—acts as a statistical fingerprint that can distinguish ordinary communication from malicious activity.

Network security is concerned with preserving the confidentiality, integrity and availability of the data and services that traverse a network. As organisations and individuals depend more heavily on networked services, the network itself has become a primary target for attackers. Network attacks take many forms, including *denial-of-service* (DoS) and *distributed denial-of-service* (DDoS) attacks that exhaust the resources of a target, *brute-force* attacks that repeatedly guess credentials for services such as SSH and FTP, *web attacks* such as SQL injection and cross-site scripting, *botnet* activity, and stealthy *infiltration* attempts. These attacks differ widely in volume, timing and purpose, which is precisely what makes them difficult to recognise with a single fixed rule.

An Intrusion Detection System (IDS) is a security mechanism that monitors a system or network for malicious activity and reports suspected intrusions. A Network Intrusion Detection System (NIDS) applies this idea at the level of network traffic: it inspects flows crossing the network and decides whether each one represents normal (benign) behaviour or an attack. Intrusion detection is traditionally approached in two ways.

Signature-based IDS compares observed traffic against a database of known attack patterns, or *signatures*. This approach is precise and produces few false alarms for attacks it already knows, but it is fundamentally reactive: it cannot detect any attack whose signature is not yet in its database, and the database must be constantly updated by human experts.

Anomaly-based IDS instead builds a model of normal behaviour and flags any traffic that deviates significantly from that model. This approach can, in principle, detect

previously unseen attacks, but defining "normal" precisely is difficult and such systems can generate a high number of false positives.

Machine-learning-based IDS generalises the anomaly-based idea by learning the distinction between benign and attack traffic directly from labelled examples, rather than from hand-written rules. Given a dataset of network flows in which each flow is described by numerical features and tagged with its true class, a classifier learns a decision function that maps a feature vector to a predicted class. Once trained, the model can classify new flows automatically and at high speed. This ability to *learn* the boundary between traffic classes, to generalise beyond exact matches, and to be retrained as new data becomes available, is what motivates the use of machine learning for intrusion detection in this project.

The importance of automated intrusion detection follows directly from the scale of modern networks. A single link can carry millions of flows in a short period, far more than any human analyst can inspect. Automation allows every flow to be examined consistently and without fatigue. Equally important is real-time network monitoring: an attack that is only discovered hours after it succeeds is of limited value to defenders, whereas a system that classifies traffic as it arrives can support timely response. The proposed NIDS combines both ideas—an accurate machine-learning classifier trained offline on a large labelled dataset, and a live pipeline that captures real traffic, converts it into the same flow features, and classifies it as it is observed, presenting the results on a monitoring dashboard.

1.2 Problem Statement

The security of computer networks is challenged by several interrelated problems that this project seeks to address:

- **Increasing volume and variety of attacks.** Networks face a continuously growing number of attacks that span many categories—volumetric DDoS floods, slow denial-of-service attacks, credential brute-forcing, web application attacks and infiltration—each with distinct traffic behaviour.
- **Large volumes of traffic.** Even a modest network produces far more flows than can be examined by hand, so any purely manual monitoring strategy fails to scale.
- **Difficulty of manual monitoring.** Recognising an attack among legitimate traffic requires expertise and constant attention; human monitoring is slow, expensive and inconsistent.
- **Need for automated classification.** An automated system is needed that can

assign every flow to a traffic class without human intervention.

- **Limitations of traditional approaches.** Signature-based detection cannot recognise attacks that are not already in its database, and simple rule-based anomaly detection is prone to high false-alarm rates.
- **Need for machine-learning-based detection.** A data-driven model that learns the distinction between benign and attack traffic can generalise beyond exact matches and be retrained as threats evolve.
- **Need for live traffic detection.** A model trained on a historical dataset is only useful in practice if it can be connected to live traffic, so that real flows are captured, transformed into features and classified as they occur.

The central problem, therefore, is to design and implement a machine-learning based NIDS that is trained on a large, realistic labelled dataset, that handles the strong class imbalance inherent in such data, and that can be applied both to uploaded datasets and to live network traffic through a usable monitoring interface.

1.3 Objectives

To develop a machine learning and deep learning based NIDS that classifies network traffic as benign or identifies its specific attack type, deployed through a web-based interface supporting live, single-flow and batch detection.

1.4 Scope and Applications

The scope of this project is the design, training and demonstration of a machine-learning based NIDS together with a supporting web application. The system is trained and evaluated on the CSE-CIC-IDS2018 dataset and is able to classify flows described by thirty CICFlowMeter-style features into fifteen traffic classes. It provides three modes of operation—classification of a single manually entered flow, classification of an uploaded dataset (which doubles as an offline evaluation when ground-truth labels are supplied), and live detection of captured traffic—all served through a single web interface.

Realistic applications of a system of this kind include:

- **Network monitoring:** continuous observation of traffic on a link to identify flows that behave like known attack categories.

- **Security monitoring:** supporting an analyst by automatically flagging and categorising suspicious traffic.
- **Educational cybersecurity environments:** demonstrating how machine learning is applied to intrusion detection, and how different attacks appear at the flow level.
- **Small network security systems:** lightweight detection for small or laboratory networks where a full commercial IDS is unnecessary.
- **Traffic analysis:** studying the statistical characteristics of benign and malicious flows.
- **Intrusion detection research:** providing a reproducible baseline for experimenting with feature selection, class balancing and model comparison.

The system is developed as an academic and laboratory-scale project. It is not claimed to be an enterprise-grade, production-hardened deployment; in particular, the live-capture component is designed for demonstration and controlled testing rather than for high-throughput operational networks. The boundaries of the work and directions for extending it are discussed further in Chapter 4.

2. Literature Review

This chapter surveys prior work relevant to the project, covering intrusion detection systems, the application of machine learning and deep learning to intrusion detection, network traffic classification, the CSE-CIC-IDS2018 dataset, anomaly and real-time detection, class imbalance, existing systems, limitations of previous systems and the solution this project provides.

2.1 Intrusion Detection Systems

Intrusion detection has been studied extensively and is broadly divided into signature-based and anomaly-based detection. Signature-based systems match traffic against a database of known attack patterns and are effective against catalogued threats but cannot recognise novel attacks. Anomaly-based systems model normal behaviour and flag deviations, offering the potential to detect unknown attacks at the cost of higher false-alarm rates. Early benchmarks such as KDD'99 and NSL-KDD were widely used to evaluate such systems but were repeatedly criticised for being unrepresentative of modern traffic, which motivated the creation of more realistic datasets that capture contemporary attacks and network conditions [1, 2].

2.2 Machine Learning and Deep Learning for Intrusion Detection

Machine learning based intrusion detection reframes detection as a supervised classification problem: a model is trained on labelled flow records so that it can assign new flows to a benign or attack class. A comprehensive systematic study by Ahmad *et al.* [3] reviewed ML and DL based NIDS published between 2017 and 2020 and found that no single approach dominates across all attack types, with class imbalance and the generalisation gap between dataset evaluation and live traffic identified as persistent open problems.

Classical ensemble methods—random forests and gradient-boosted trees—perform strongly on flow-based tabular data and are consistently among the top performers in intrusion-detection benchmarks [4, 2]. Maseer *et al.* [2] benchmarked ten popular supervised and unsupervised ML algorithms on CICIDS2017 and found that tree-based ensembles achieve the best balance of accuracy, precision and recall for anomaly detection. Tama and Lim [4] conducted a systematic mapping study across 124

publications and introduced a stacked ensemble that combines multiple ensemble learners, showing significant performance improvements over individual models.

Deep learning approaches have also been extensively explored. Vinayakumar *et al.* [5] proposed a scalable deep neural network (DNN) framework that detects and classifies attacks at the network and host level across multiple publicly available datasets, demonstrating that deep networks can match or exceed classical classifiers when sufficient training data is available. Wang *et al.* [6] introduced an ensemble feature-selection approach combined with a DNN that reduces the feature space while maintaining strong detection performance on the CSE-CIC-IDS2018 dataset. Kocher and Kumar [7] survey both ML and DL methods for IDS and conclude that while deep networks offer representational power for complex feature interactions, tree-based ensembles remain competitive on structured tabular traffic features—a finding directly confirmed by the results of this project. Hyperparameter optimisation significantly influences the performance of all these models, and Optuna [8] is adopted in this project to tune all candidate models under a common automated search budget.

2.3 Network Traffic Classification

Flow-based traffic classification represents each connection by a fixed vector of statistical features—counts and sizes of packets, flow duration, inter-arrival times and transport-layer flag counts—rather than inspecting packet payloads. This approach scales to high traffic volumes and is robust to payload encryption. The CSE-CIC-IDS2018 dataset uses CICFlowMeter-derived features that capture the timing and volume behaviour of bidirectional flows [1]. Because payloads are not required, flow-based classification is well suited both to offline analysis of large captured datasets and to live monitoring, where flows are assembled from packets in real time. Maseer *et al.* [2] confirm that such flow-based feature sets provide sufficient discriminative information for accurate multi-class attack detection.

2.4 The CSE-CIC-IDS2018 Dataset

The CSE-CIC-IDS2018 dataset was produced by the Communications Security Establishment and the Canadian Institute for Cybersecurity [1]. It captures benign traffic together with a broad range of contemporary attacks—including brute-force, denial-of-service, distributed denial-of-service, web attacks, infiltration and botnet activity—over multiple capture days, with more than eighty flow features extracted per record. Its scale and diversity have made it a standard benchmark for machine-learning intrusion

detection research.

Leevy and Khoshgoftaar [9] reviewed a large number of models trained on CSE-CIC-IDS2018 and noted that only about one sixth of the traffic is malicious, that many reported scores are unexpectedly high and may reflect overfitting, and that most studies neither adequately address class imbalance nor document their data-cleaning steps. Liu *et al.* [10] further show that both CIC-IDS-2017 and CSE-CIC-IDS-2018 contain labelling and feature-extraction errors, underscoring the importance of explicit data cleaning. Wang *et al.* [6] and Karatas *et al.* [11] both use CSE-CIC-IDS2018 as their primary evaluation benchmark, further establishing its role as the standard dataset for this class of research.

2.5 Anomaly Detection and Real-Time Intrusion Detection

While much of the literature evaluates models offline on static datasets, the practical value of an intrusion detector depends on its ability to operate on live traffic. Real-time detection requires that packets be captured, assembled into flows and converted into the same features used for training, all within a processing pipeline that keeps pace with the traffic. Ahmad *et al.* [3] identify the gap between controlled dataset conditions and live traffic as one of the most important open challenges in the field. A model that performs well offline can degrade if the features computed at run time do not match those seen during training. This project addresses that concern explicitly by reusing a single feature and preprocessing pipeline for both offline evaluation and live detection, ensuring that a live flow is described identically to a dataset flow.

2.6 Class Imbalance in Intrusion Detection

Intrusion-detection datasets are severely imbalanced: benign traffic vastly outnumbers attacks, and some attack classes contain only a handful of examples. On such data a classifier can achieve very high accuracy while failing on exactly the rare classes that matter most. Karatas *et al.* [11] demonstrate on CSE-CIC-IDS2018 that explicitly addressing imbalance—through oversampling and class weighting—measurably improves the detection of minority attack classes while keeping overall accuracy high. Generative adversarial approaches such as CTGAN [12] learn to synthesise realistic tabular records for rare classes and have become a practical tool for augmenting low-count classes in intrusion-detection research. Leevy and Khoshgoftaar [9] note that most prior studies on CSE-CIC-IDS2018 do not adequately report how imbalance is handled, which undermines reproducibility and inflates reported scores. This project

combines capping of the dominant benign class with CTGAN-based augmentation of rare attack classes, and evaluates models primarily by macro F1 to give every class equal weight.

2.7 Existing Systems

Existing network intrusion detection systems can be grouped into three broad categories. The first is signature-based systems like Snort, which match live traffic against a maintained database of known attack signatures. Such systems are accurate for catalogued attacks but cannot detect novel or zero-day attacks and require continuous manual updates. The second is classical ML systems trained on older benchmarks (KDD'99, NSL-KDD), which demonstrated the viability of learning-based detection but were built on data no longer representative of modern traffic [1]. The third and most relevant category is modern ML and DL systems trained on the CIC datasets. Ahmad *et al.* [3] provide a systematic taxonomy of such approaches, while Vinayakumar *et al.* [5] demonstrate that deep neural networks can perform effective multi-class traffic classification across several datasets. Wang *et al.* [6] show that ensemble feature selection combined with a DNN achieves strong results specifically on CSE-CIC-IDS2018, and Tama and Lim [4] confirm that stacked ensembles consistently outperform single classifiers. The present project belongs to this third category and additionally couples the offline detector to a live-capture pipeline.

2.8 Limitations of Previous Systems

Despite their strengths, the existing approaches have well-documented limitations that this project seeks to overcome:

- **Inability to detect novel attacks.** Signature-based systems can only detect attacks whose signatures are already in their database and must be continually updated by experts, so they are ineffective against new or modified attacks.
- **Binary classification only.** Many existing ML-based NIDS [2, 4] frame detection as a two-class problem—benign versus malicious—which is insufficient for operational use. A security analyst needs to know not only *that* an attack is occurring but *which specific attack type* it is (e.g. DDoS, brute-force, SQL injection, botnet) in order to select an appropriate response. Systems that collapse all attacks into a single "attack" label provide no such actionable information.
- **Misleading accuracy under class imbalance.** Leevy and Khoshgoftaar [9] find that many ML studies on CSE-CIC-IDS2018 report unexpectedly high accuracy

that is inflated by the dominance of benign traffic, and that rare attack classes are handled poorly even when headline accuracy is excellent.

- **Weak data cleaning and reproducibility.** Liu *et al.* [10] show that the CIC datasets contain labelling and feature-extraction errors, and that inadequately documented preprocessing procedures undermine reproducibility.
- **Offline-only evaluation.** Ahmad *et al.* [3] note that the large majority of prior systems are evaluated only on static labelled datasets and are never connected to live traffic, leaving open the question of whether they work on real captured flows.

2.9 Solution Provided

The system developed in this project directly addresses each of the limitations identified above:

- **Learning-based detection of novel attacks.** A trained ML and DL classifier learns the distinction between benign and attack traffic from data and can generalise beyond exact signature matches, and can be retrained as new threats emerge.
- **Fine-grained multi-class attack identification.** Rather than producing a simple benign/attack binary verdict, the system classifies every network flow into one of fifteen specific traffic classes—including FTP-BruteForce, SSH-BruteForce, DDoS-HOIC, DDoS-LOIC-HTTP, DoS-Hulk, DDoS-LOIC-UDP, DoS-GoldenEye, DoS-Slowloris, DoS-SlowHTTPTest, Bot, Infilteration, Brute Force-Web, Brute Force-XSS, SQL Injection, and Benign. This fine-grained classification gives a security analyst directly actionable information about the nature of the threat, enabling a targeted and proportionate response rather than a generic alert.
- **Principled class balancing and honest evaluation.** The dominant benign class is capped and rare attack classes are augmented with CTGAN [12], and models are judged by macro F1 so that performance on rare attack types—including the least-common SQL Injection class—receives equal weight.
- **Explicit, documented data preparation.** The raw dataset is cleaned and de-duplicated through a clearly specified pipeline, directly addressing the data-quality concerns of Liu *et al.* [10] and Leevy and Khoshgoftaar [9].
- **Live detection with specific attack labelling.** The offline-trained model is connected to a live-capture pipeline that reuses the same thirty-feature representation and preprocessing. When a live flow is classified, the dashboard displays not just "attack" but the specific predicted attack category, and its ability to do so correctly

is confirmed by a controlled live-attack demonstration.

The reviewed literature establishes flow-based machine learning and deep learning as effective and widely studied approaches to network intrusion detection. Ensemble tree methods and deep neural networks are the strongest candidate model families, and their combination—as implemented in this project—is well supported by the evidence. Three recurring pitfalls stand out: the reduction of detection to a binary benign/attack label that provides no actionable information about attack type, inadequate handling of class imbalance, and insufficient attention to data quality and reproducibility. The gap between offline evaluation and live operation is also a major concern. The methodology described in the next chapter is designed with all these lessons in mind: it performs explicit data cleaning, mutual-information feature selection, principled class balancing, and tuned model comparison judged on macro F1, and delivers a shared pipeline that classifies live traffic into one of fifteen specific traffic classes rather than a generic binary verdict.

3. Methodology

This chapter describes how the Network Intrusion Detection System was designed, implemented and validated. It follows the development process model, presents the overall system block diagram together with the dataset, preprocessing, feature selection, class balancing, models, training, evaluation and live-detection pipeline, and then documents the algorithms, flowchart, UML diagrams, tools used, and verification and validation of the system.

3.1 Process Model

The project was developed using an iterative and incremental process model. This model was chosen because it fits the exploratory, data-driven nature of the work: the machine-learning and deep learning pipeline was built in stages, and each stage was evaluated and refined before the next was added, rather than being specified completely in advance. The phases of the model and the way they repeat are shown in Figure 3.1.

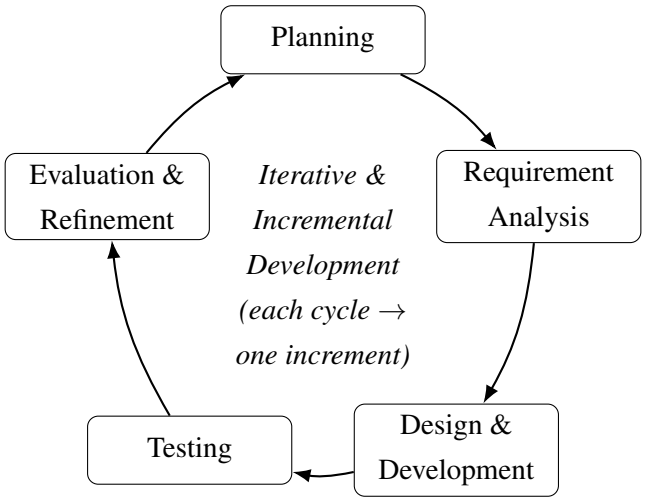

Figure 3.1: Iterative and Incremental Process Model

The work proceeded through repeated iterations of the five phases shown in Figure 3.1. Each phase plays the following role:

- **Planning:** the goal of the current iteration is defined—for example, preparing the data, selecting features, building a model, or adding a part of the web application—together with the criteria for judging whether it succeeds.

- **Requirement Analysis:** the specific inputs and constraints of that goal are examined, such as which data, features, libraries or interface behaviour the increment requires.
- **Design & Development:** the corresponding part of the system is designed and implemented, for instance the preprocessing steps, a classifier, or a page of the interface.
- **Testing:** the newly built increment is exercised to confirm that it behaves as intended before it is relied upon.
- **Evaluation & Refinement:** the outcome of the increment is assessed, and the lessons learned are carried into the planning of the next iteration.

Each complete pass through these phases delivers a working, testable increment of the system, and the cycle repeats until the full pipeline—from data preparation through model training to live detection—is built.

The iterative model was appropriate because early increments (for example the choice of preprocessing and feature set) directly shaped later ones (the models and their evaluation), and because findings at the evaluation stage—notably the importance of the rare classes—fed back into decisions about class balancing and model selection.

3.2 System Block Diagram

The system has two connected halves: an offline pipeline that prepares data and trains the detection model, and an online pipeline that applies the trained model to new flows. The online pipeline supports three input modes through the web application: *single-flow classification* (a manually entered flow), *dataset (batch) classification* (an uploaded CSV file), and *live network traffic detection* (real-time packet capture). All three modes share the same feature representation, preprocessing and prediction pipeline. The overall architecture is shown in Figure 3.2.

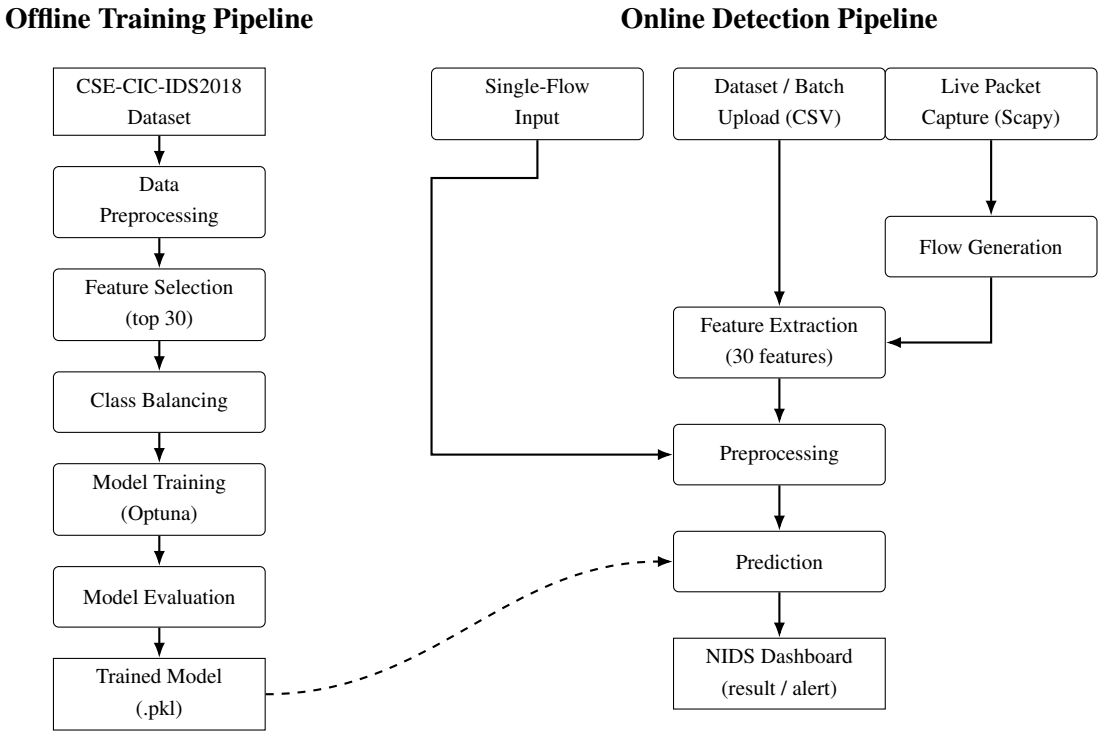

Figure 3.2: System Architecture

3.2.1 Dataset

The project uses the CSE-CIC-IDS2018 dataset, produced by the Communications Security Establishment and the Canadian Institute for Cybersecurity. The raw dataset consists of labelled network-flow records captured over several days, with each flow described by more than eighty CICFlowMeter features. After cleaning and de-duplication, the working dataset contained approximately 15.8 million flow records. The data spans fifteen traffic classes: one benign class and fourteen attack types. The class distribution is extremely imbalanced, as summarised in Table 3.1: benign traffic accounts for the overwhelming majority of records, while some attack classes (for example SQL Injection) contain fewer than a hundred examples.

Table 3.1: CSE-CIC-IDS2018 Class Distribution

Encoded	Class	Records
0	Benign	13,446,845
4	DDOS attack-HOIC	668,461
6	DDoS attacks-LOIC-HTTP	576,175
8	DoS attacks-Hulk	434,873
1	Bot	282,310
12	Infilteration	161,897
14	SSH-Bruteforce	117,322
7	DoS attacks-GoldenEye	41,455
11	FTP-BruteForce	39,352
9	DoS attacks-SlowHTTPTest	19,462
10	DoS attacks-Slowloris	10,285
5	DDOS attack-LOIC-UDP	1,730
2	Brute Force -Web	611
3	Brute Force -XSS	230
13	SQL Injection	87
	Total	**15,801,095**

Because training on the full multi-million-row dataset is unnecessary and computationally wasteful, a stratified modeling subset of roughly one million rows was drawn from the cleaned data. Every row of the rare and minority classes was retained, while the dominant classes (particularly benign) were capped using reservoir sampling so that nothing informative was discarded but the majority classes did not overwhelm the sample. This subset was reduced to the selected features and split into training and test sets.

3.2.2 Data Preprocessing

The raw CSV files were cleaned in a single chunked pass per file to keep memory use bounded. The preprocessing pipeline performs the following steps, all of which are actually implemented in the data-preparation notebook:

- **Column-name normalisation:** leading and trailing whitespace is stripped from every column name so that headers are consistent across files.
- **Identifier removal:** non-generalisable identifier columns (`Flow ID`, `Src IP`, `Src Port`, `Dst IP`), which appear only in some files, are dropped.

- **Numeric coercion:** every non-label column is coerced to a numeric type, which also repairs the small number of corrupted rows.
- **Infinity handling:** positive and negative infinities are replaced with missing values.
- **Missing-value imputation:** remaining missing values are imputed with the column median.
- **De-duplication:** exact duplicate rows are removed, first within each chunk and then globally across all cleaned chunks using an out-of-core merge.

The output is a single cleaned dataset from which the stratified modeling subset is built. This explicit and reproducible cleaning procedure directly addresses the data-quality concerns.

3.2.3 Feature Selection

The cleaned records contain far more features than are needed, and many carry little discriminative information. Feature selection was performed using mutual information, which measures the statistical dependence between each feature and the class label. For a feature X and the label Y, the mutual information is

$$I(X;Y) = \sum_{y \in Y} \sum_{x \in X} p(x,y) \log\left(\frac{p(x,y)}{p(x)\,p(y)}\right), \tag{3.1}$$

where $p(x,y)$ is the joint distribution of the feature and the label and $p(x)$, $p(y)$ are their marginals. A mutual information of zero indicates independence (the feature carries no information about the class), while larger values indicate stronger dependence. The scores were computed on the training split with a k-nearest-neighbour estimator, and the top thirty features by mutual information were retained. Figure 3.3 shows the mutual information scores of the top twenty features; the remaining ten selected features continue the same trend of diminishing scores down to rank 30.

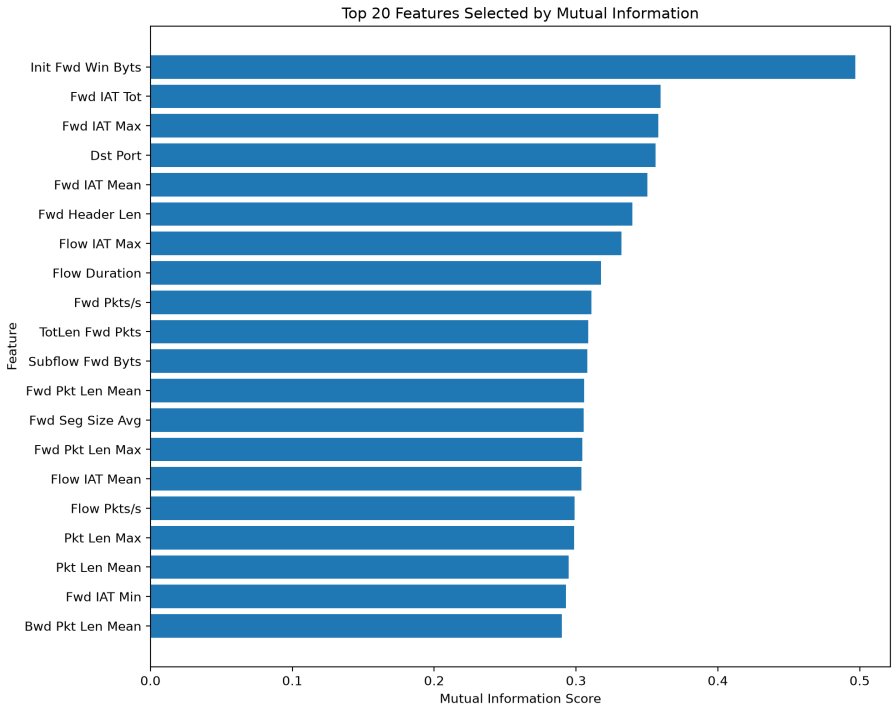

Figure 3.3: Mutual Information Scores of Selected Features

The thirty selected features are dominated by directional inter-arrival-time statistics, packet- and byte-length statistics, flow-rate features, the destination port, and TCP window and header sizes—quantities that capture the timing and volume signatures by which attack traffic differs from benign traffic.

3.2.4 Class Balancing

The strong class imbalance of the dataset (Table 3.1) was addressed on the *training set only*, leaving the test set untouched so that evaluation reflects the true class distribution. Two complementary techniques were applied:

- **Capping the dominant class:** the benign class was reduced to 100,000 rows, and other very large classes were left at their natural counts, so that the majority classes no longer dominated training.
- **Synthetic augmentation of rare classes:** the rare attack classes were augmented up to target counts using a Conditional Tabular GAN (CTGAN). For the ultra-rare classes with too few real samples for a generative model to learn a stable distribution, bootstrap oversampling with small Gaussian jitter (proportional to each feature's standard deviation) was used instead, which stays closer to the real distribution than a generative model trained on a handful of rows.

The effect of balancing on the training-set class distribution is shown in Figure 3.4. Classes such as SQL Injection and Bot, whose real counts were already deemed adequate or which were kept as-is, were not augmented, while the smallest web-attack and slow-DoS classes were raised to the low tens of thousands. After balancing, the training set contained 281,295 rows; the untouched test set contained 200,498 rows.

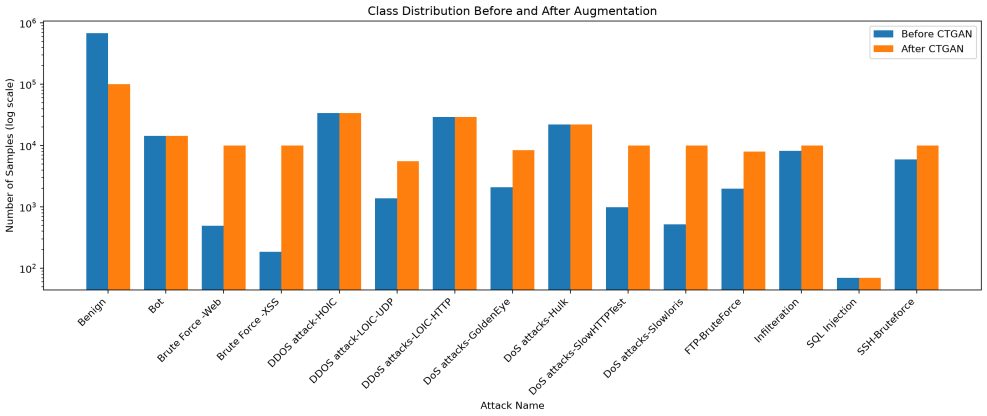

Figure 3.4: Class Distribution Before and After Balancing

3.2.5 Machine Learning Models

Four classifiers were trained and compared. Three are decision-tree ensembles selected as the strongest performers from a wider Optuna search over five tree-based algorithms, and the fourth is a neural network trained separately.

3.2.5.1 Histogram-based Gradient Boosting (HistGradientBoosting)

Gradient boosting builds an additive ensemble of decision trees in which each new tree is fitted to the negative gradient of the loss of the current ensemble:

$$F_m(x) = F_{m-1}(x) + \nu\, h_m(x), \tag{3.2}$$

where F_m is the ensemble after m trees, h_m is the newly added tree, and ν is the learning rate. The histogram-based variant bins continuous features into a small number of integer-valued histograms before splitting, which makes training substantially faster and more memory-efficient on large datasets without materially reducing accuracy.

Advantages: very fast to train, handles large tabular data well, and strong on the rare classes.
Limitations: as with all boosting, it can overfit if not regularised.

Why considered: it gave the best macro F1 of all models and is used as the default detection model.

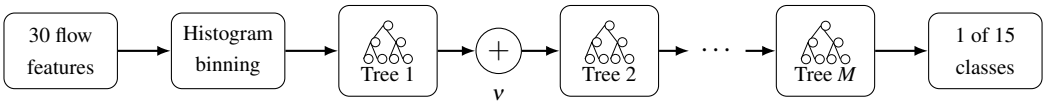

Figure 3.5: HistGradientBoosting Model

3.2.5.2 XGBoost (Extreme Gradient Boosting)

XGBoost is a scalable, regularised implementation of gradient boosting. It optimises a regularised objective

$$\mathscr{L} = \sum_i \ell\big(y_i, \hat{y}_i\big) + \sum_k \Omega\big(f_k\big), \qquad \Omega(f) = \gamma T + \tfrac{1}{2}\lambda \|w\|^2, \tag{3.3}$$

where ℓ is the training loss, Ω penalises the complexity of each tree f_k (with T leaves and leaf weights w), and γ, λ control regularisation.

Advantages: highly accurate and well regularised; achieved the best raw accuracy of the four models.
Limitations: more hyperparameters to tune and slightly weaker than HistGradient-Boosting on the rarest classes.
Why considered: it is a standard high-performance baseline for tabular classification and is offered as a selectable model in the web app.

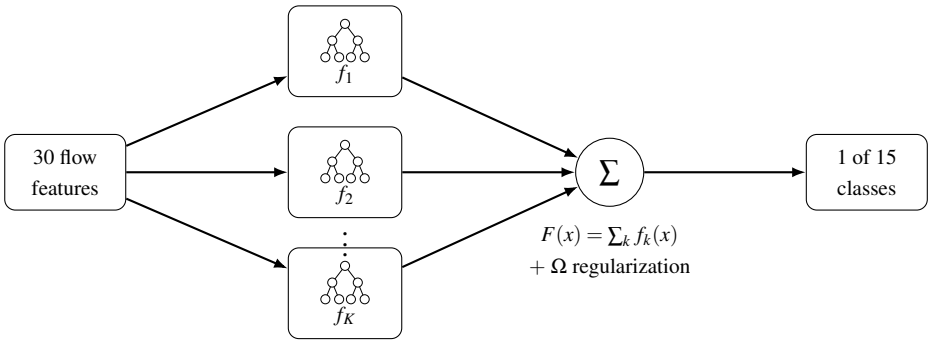

Figure 3.6: XGBoost Model

3.2.5.3 Random Forest

A random forest trains many decision trees on bootstrap samples of the data, each using a random subset of features at every split, and combines their predictions by majority vote:

$$\hat{y} = \text{mode}\left\{ T_1(x), T_2(x), \ldots, T_B(x) \right\}, \tag{3.4}$$

where T_b is the b-th tree. Averaging over decorrelated trees reduces variance and improves generalisation.

Advantages: robust, easy to train, and provides a strong baseline.
Limitations: it was the weakest of the four models on this dataset and its trained model file was not deployed.
Why considered: it is a well-understood reference model and was included in the model comparison and the data-scaling study.

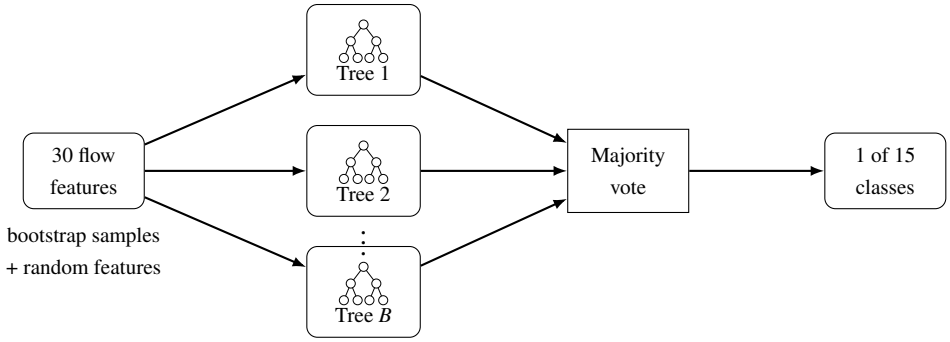

Figure 3.7: Random Forest Model

3.2.5.4 Multi-Layer Perceptron (MLP)

The MLP is a feed-forward neural network. Each layer applies an affine transform followed by a non-linear activation,

$$h^{(l)} = \phi\left(W^{(l)} h^{(l-1)} + b^{(l)}\right), \tag{3.5}$$

with a softmax output layer producing class probabilities and training minimising the cross-entropy loss. Because neural networks are sensitive to feature scale, the MLP is trained on standardised features (zero mean, unit variance) using a scaler that is saved alongside the model and reapplied at prediction time.

Advantages: can model non-linear feature interactions and serves as a deep-learning baseline.
Limitations: by far the slowest to train and the weakest on macro F1, indicating comparatively poor rare-class performance.
Why considered: it provides a neural-network point of comparison against the tree ensembles.

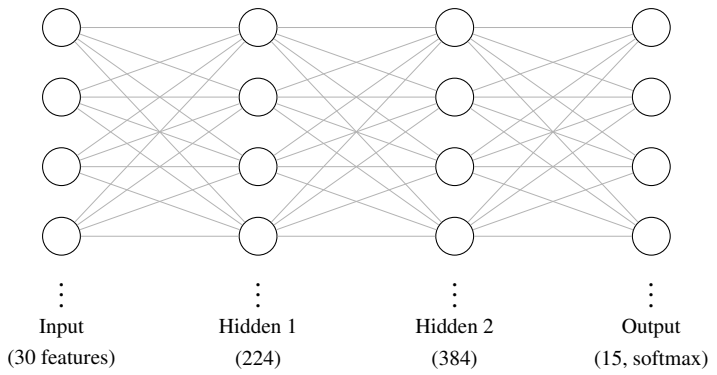

Figure 3.8: Multi-Layer Perceptron (MLP) Model

The tuned hyperparameters of each model, selected by Optuna, are summarised in Table 3.2.

Table 3.2: Tuned Hyperparameters of Each Model

Model	**Selected hyperparameters**
HistGradientBoosting	max_iter = 250, learning_rate = 0.186, max_depth = 5, max_leaf_nodes = 38, l2_regularization = 0.970
XGBoost	n_estimators = 450, max_depth = 7, learning_rate = 0.056, subsample = 0.799, colsample_bytree = 0.809, min_child_weight = 5, gamma = 2.605
Random Forest	n_estimators = 500, max_depth = 30, min_samples_split = 3, min_samples_leaf = 1, max_features = $\log_2$, class_weight = balanced
MLP	hidden layers = $(224, 384)$, activation = ReLU, alpha = 1.7×10^{-4}, learning_rate_init = 8.8×10^{-4}, batch_size = 64

3.2.6 Model Training

Model development followed the pipeline

$$\text{Dataset} \rightarrow \text{Preprocessing} \rightarrow \text{Feature Selection} \rightarrow \text{Train/Test Split} \rightarrow \text{Class Balancing} \rightarrow \text{Model Training} \rightarrow \text{Validation} \rightarrow \text{Model Saving}.$$

The stratified modeling subset was split into training and test sets using an 80/20 stratified split (random state 42), preserving the class proportions in both sets. Feature

selection and class balancing were applied to the training set as described above; the test set was left unbalanced so that reported scores reflect real class frequencies.

Hyperparameters were optimised using a tree-structured Parzen estimator sampler and a median pruner. A unified search over five tree-based algorithms (decision tree, random forest, histogram gradient boosting, XGBoost and AdaBoost) was run for 60 trials with 3-fold stratified cross-validation on a 100,000-row search sample, optimising the macro F1-score. The three best algorithms were then retrained on the full balanced training set, and the MLP was tuned separately in its own Optuna study. The final trained models were serialised to disk (with the MLP's feature scaler) for use by the web application.

3.2.7 Model Evaluation

Models were evaluated on the held-out test set using standard multi-class classification metrics. For each class, TP, FP, FN and TN denote true positives, false positives, false negatives and true negatives respectively. The four metrics are defined as follows:

$$\text{Precision} = \frac{TP}{TP + FP} \tag{3.6}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{3.7}$$

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{3.8}$$

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{3.9}$$

The macro F1-score is the unweighted mean of the per-class F1-scores and therefore gives every class equal importance regardless of its frequency. It is adopted as the primary metric because the dataset is heavily imbalanced: a model can reach very high accuracy while performing poorly on the rare attack classes, so accuracy alone is misleading. The confusion matrix is used to inspect which classes are confused with which. The numerical results are presented and discussed in Chapter 4.

3.2.8 Live Detection Pipeline

The trained offline model is connected to live traffic through a dedicated capture pipeline. The pipeline is deliberately identical in its feature and preprocessing stages to the offline pipeline, so that a live flow is represented by exactly the same thirty features, in the same order, as a dataset flow. The stages are:

1. **Packet capture:** packets are captured from a network interface (or replayed from a stored capture file) using Scapy.
2. **Flow generation:** captured packets are grouped into bidirectional flows keyed by source/destination address, port and transport protocol; a flow is finalised when it has been idle beyond a fixed timeout.
3. **Feature extraction:** for each finalised flow the thirty flow features are computed from the accumulated packet statistics, reproducing the CICFlowMeter definitions used to build the training data.
4. **Preprocessing:** the feature vector is assembled into a one-row frame with the exact training column order, and (for the neural-network model) standardised with the saved scaler.
5. **Prediction:** the feature vector is passed to the trained classifier, which outputs the predicted class and its probability.
6. **Dashboard:** the prediction is displayed on the live monitoring dashboard, where benign and attack flows are shown as they are classified.

Using a single shared feature and preprocessing pipeline is the key design decision that lets a model trained entirely on the offline dataset be applied directly to live traffic without retraining, and ensures that a live flow is in-distribution for the model.

3.2.9 Web Application

The system is delivered as a Django web application with HTML/CSS/JavaScript front end. It exposes the detection capability through several pages that all sit on top of the same prediction module:

- **Single-flow classification:** a form for the thirty features (grouped into logical sections, each field pre-filled with the dataset median as a placeholder) that returns the predicted class, its confidence and the three most likely classes.
- **Dataset (batch) classification:** an uploaded CSV file is classified row by row. A flexible column-mapping layer accepts alternative CICFlowMeter naming con-

ventions and headers with stray spaces. If the file contains a ground-truth `Label` column, the page additionally reports accuracy, macro F1 and per-class metrics, so an upload doubles as an offline evaluation.

- **Live monitoring:** starts and stops live capture and streams classified flows to the dashboard.
- **History:** every run is appended to a log and can be reviewed and exported as a spreadsheet.

A JSON API endpoint for single-flow prediction is also provided. The application is deployed to a cloud platform for online access.

3.3 Algorithm

This section presents a generalized algorithm that summarises the overall working of the Network Intrusion Detection System at a high level, covering both the offline training stage and the online detection stage. The detailed, implementation-level steps are described in the preceding sections; the algorithm below captures the essential logic without reproducing them line by line.

Algorithm 1 Generalized NIDS Algorithm

Stage 1: Offline Training

collect and clean the CSE-CIC-IDS2018 dataset
select the most informative flow features and balance the classes
train and tune the machine-learning models
select the best model and save it

Stage 2: Online Detection (three input modes)

load the trained model
Mode A — Single-Flow Classification:
    receive the 30 feature values entered manually by the analyst
    apply preprocessing and classify; display the predicted class and confidence
Mode B — Dataset / Batch Classification:
    receive an uploaded CSV file of flow records
    for each row: extract features, apply preprocessing, classify
    display per-row predictions and overall evaluation metrics
Mode C — Live Network Traffic Detection:
for each packet captured from the network interface **do**
    group packet into a bidirectional flow (timeout-based)
    if flow is complete **then**
        extract the 30 flow features
        apply the preprocessing pipeline
        classify the flow with the trained model
        display the predicted class (Benign or specific attack type) on the dashboard
    end if
end for

3.4 Flowchart

Figure 3.9 shows the complete operational flowchart of the NIDS, covering all three detection modes and the shared prediction pipeline.

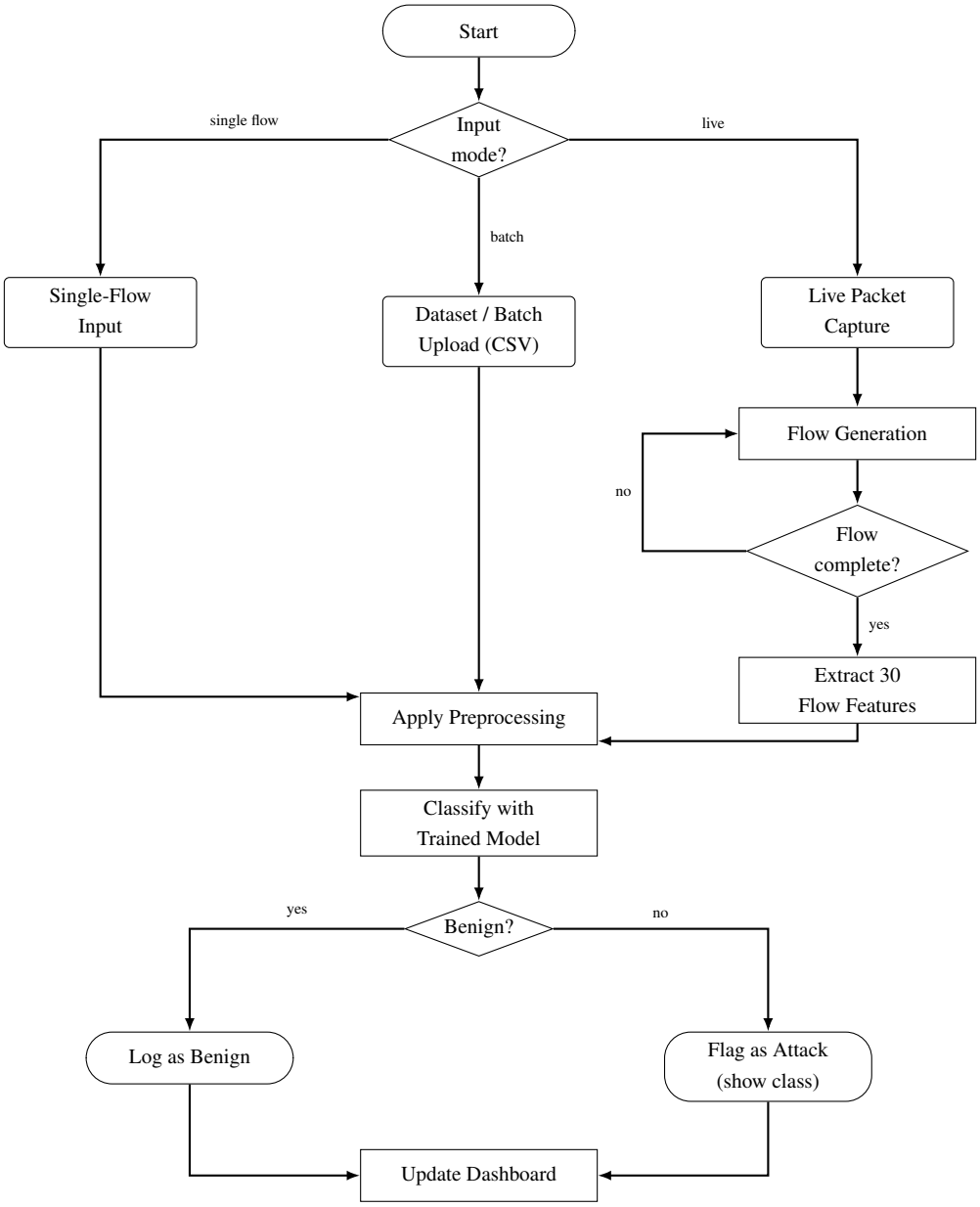

Figure 3.9: NIDS Operational Flowchart

3.5 UML Diagrams

This section presents the UML diagrams that describe the structure and behaviour of the system: a use-case diagram, an activity diagram, and the data flow diagrams.

3.5.1 Use Case Diagram

Figure 3.10 shows the main use cases available to the two actors of the system: a *Security Analyst* (a human user) and the *Network Interface* (the source of live packets).

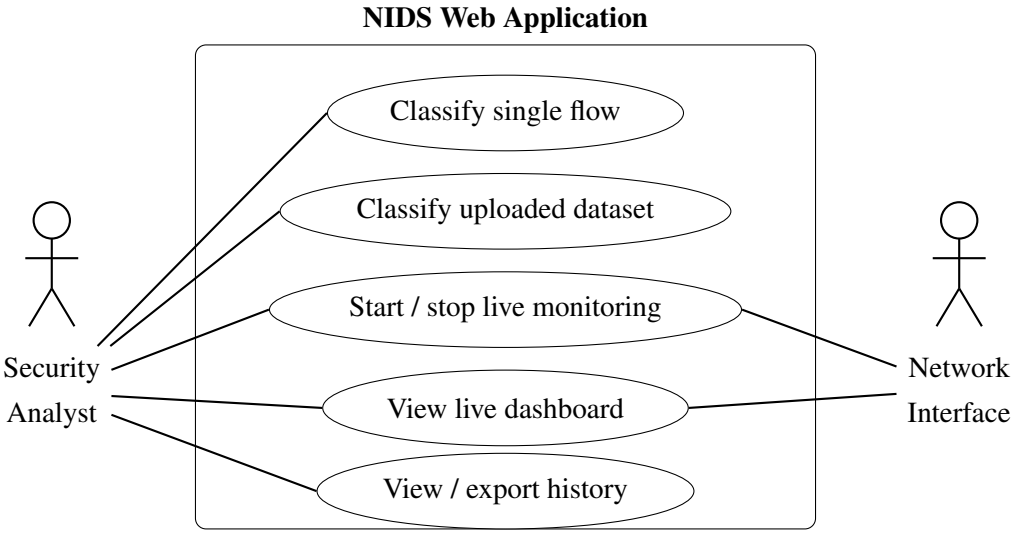

Figure 3.10: Use Case Diagram

3.5.2 Activity Diagram

Figure 3.11 shows the activity flow for all three detection modes supported by the NIDS web application.

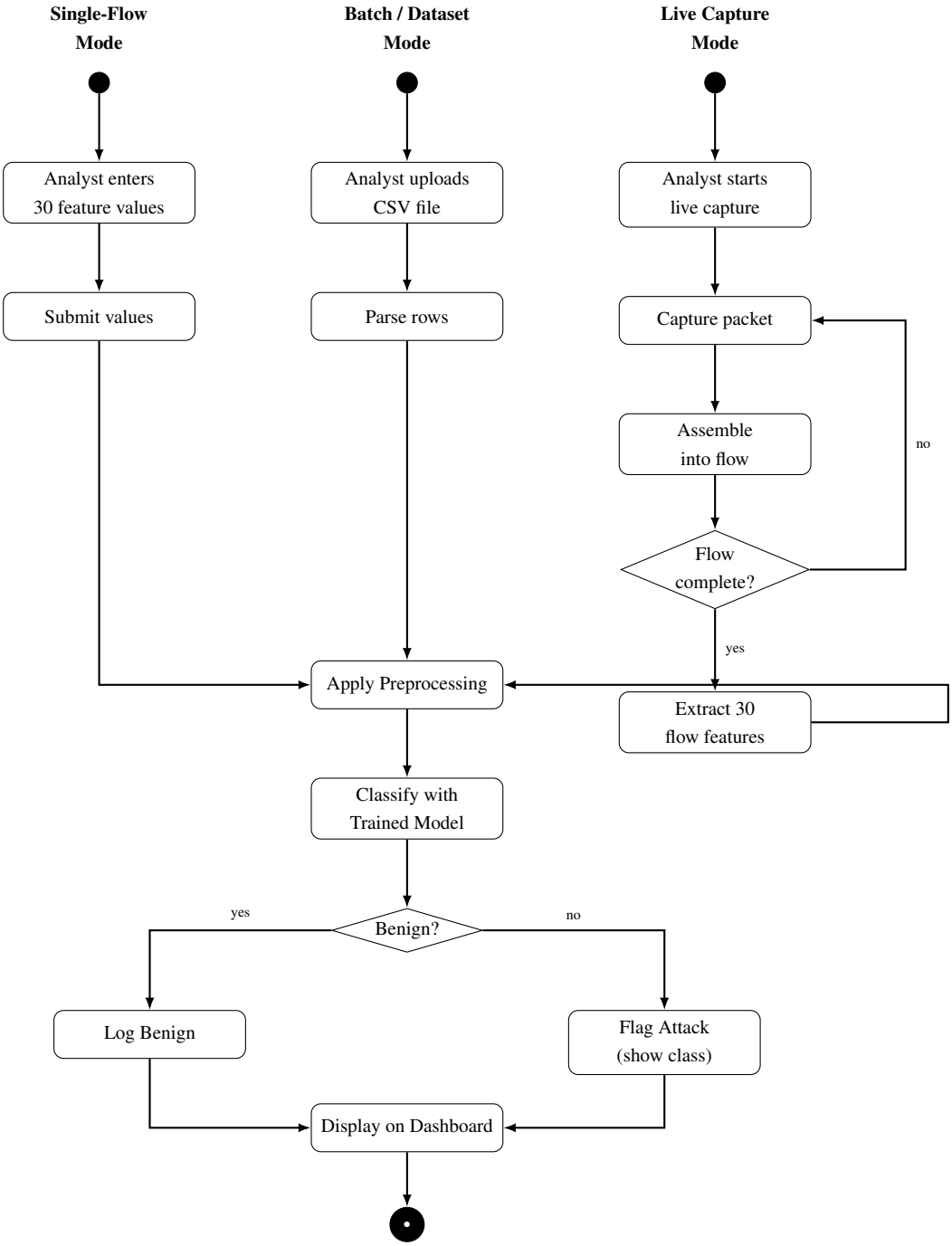

Figure 3.11: Activity Diagram — All Three Detection Modes

3.5.3 Data Flow Diagrams

The data flow diagrams (DFDs) describe how data moves through the system. The context (Level 0) diagram in Figure 3.12 treats the whole NIDS as a single process interacting with two external entities: the *Security Analyst*, who submits flows or datasets and views results, and the *Network Interface*, which supplies live packets. The Level 1 diagram in Figure 3.13 decomposes that single process into the main sub-processes and shows the two data stores (the trained model and the prediction history).

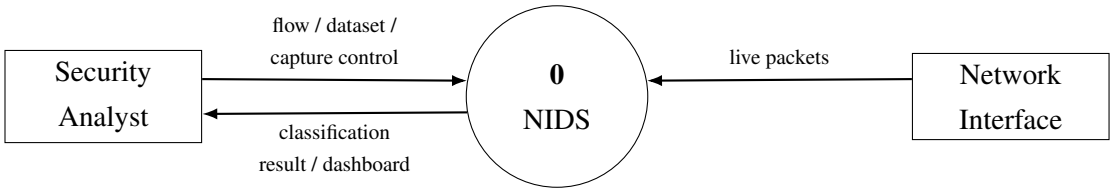

Figure 3.12: DFD Level 0 — Context Diagram

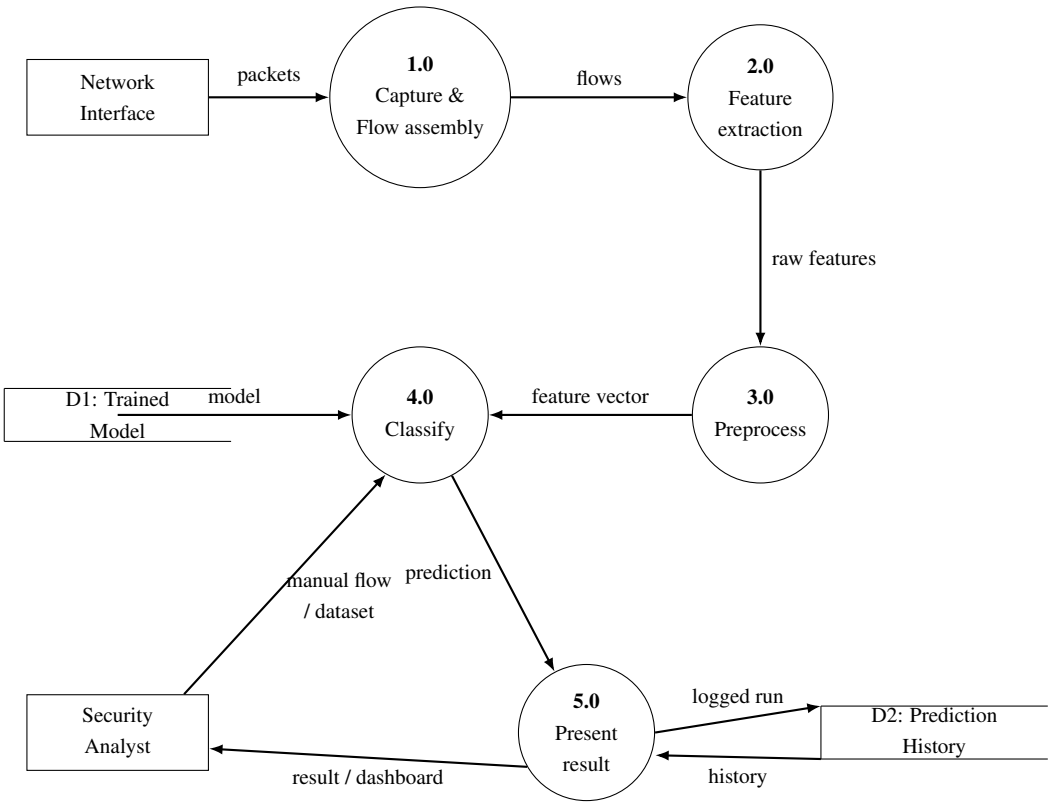

Figure 3.13: DFD Level 1 — System Processes

3.6 Tools Used

The technologies and libraries actually used in the project are listed in Table 3.3. Every item was actually used in the project's implementation.

Table 3.3: Tools and Technologies

Category	Tools / Libraries
Programming language	Python
Data processing	Pandas, NumPy, PyArrow, DuckDB
Machine learning	scikit-learn, XGBoost, Optuna
Data augmentation	CTGAN (Conditional Tabular GAN)
Visualisation	Matplotlib, Plotly
Explainability (analysis)	SHAP
Live packet capture	Scapy
Web framework	Django
Web front end	HTML, CSS, JavaScript
Deployment / serving	Gunicorn, WhiteNoise, Render
Development tools	Jupyter Notebook, VS Code
Testing	Pytest, Django test framework

3.7 Verification and Validation

Verification and validation were carried out at multiple levels throughout the development of the NIDS, covering the preprocessing pipeline, the trained models, the web application, and the live-detection workflow.

Verification

Verification is concerned with whether the system was built correctly—that is, whether each component behaves as designed.

Preprocessing pipeline consistency. The preprocessing steps applied during training (column-name normalisation, identifier removal, numeric coercion, infinity handling, missing-value imputation, and de-duplication) were verified to produce identical transformations when applied to new data at prediction time. The column-mapping layer of the web application was specifically tested to confirm that alternative CICFlowMeter column naming conventions and headers with leading whitespace are correctly normalised before any prediction is made.

Feature ordering and model input. The thirty selected features are passed to each model in the exact order established during training. This was verified by confirming that the feature list stored in the trained model metadata matches the column order of the input frame constructed at prediction time in both the batch and live pipelines.

Model loading and prediction. The serialised model files are loaded at startup and cached. This was verified by running single-flow predictions through the web interface and confirming that the predicted class, confidence score, and top-three class rankings are returned correctly and consistently.

Web application component integration. The single-flow prediction form, the batch upload endpoint, and the live-monitoring page were each tested to confirm that they interact correctly with the underlying ML prediction module. The batch upload feature was verified by checking that uploading the same labelled file with and without ground-truth labels yields byte-identical predictions, confirming that the label column is never used as a model input.

Live pipeline verification. The end-to-end live-detection path—packet capture via Scapy, flow construction with timeout-based expiry, feature extraction, preprocessing, and prediction—was verified using a synthetic capture replayed through the pipeline. The feature values computed by the live pipeline were compared against the ranges seen in the training data to confirm that live-computed features fall within expected bounds.

Validation

Validation is concerned with whether the system meets its intended objectives— that is, whether it correctly detects and classifies network intrusion traffic.

Held-out dataset evaluation. All four models were evaluated on the untouched held-out test set of 200,498 flows. Per-class precision, recall and F1-score were recorded for all fifteen traffic classes. The macro F1-score was used as the primary validation metric because the test set retains the natural class imbalance of the dataset, and macro F1 gives equal weight to every class including the rare attack types that matter most for intrusion detection. The HistGradientBoosting model achieved the best macro F1-score of 0.8627 at 98.03% accuracy and was selected as the default detection model on this basis.

Per-class and rare-class validation. The confusion matrix and per-class metrics confirmed that the system correctly identifies the majority of attack classes. High-volume classes such as DDoS-HOIC, DoS-Hulk, Bot and SSH-BruteForce reached near-perfect F1-scores, while the rare classes—Infilteration and SQL Injection—were identified as the most challenging, consistent with their very low sample counts even after balancing. This validates that the system meets its objective of fine-grained multi-class attack identification, while also highlighting where further improvement is needed.

Live traffic validation with a controlled attack. To validate the complete end-to-end detection pipeline on real network traffic, a controlled brute-force attack was generated in a local test environment. The resulting network traffic was captured by the live-detection pipeline, passed through flow construction, feature extraction and preprocessing, and classified by the trained model. The traffic was correctly identified as an attack, demonstrating that the system can successfully detect a real attack in a live setting. This confirms that the offline-trained model generalises to actual captured traffic and that the full pipeline from packet capture to dashboard display operates as intended.

Taken together, these verification and validation activities provide evidence that the implemented NIDS correctly classifies network flows into benign and specific attack categories, that its live detection pipeline operates end-to-end on real traffic, and that the system satisfies the detection and operational objectives for which it was designed.

4. Epilogue

4.1 Results and Conclusion

This section presents the results of the project, drawn from the actual model evaluations produced during the project, and distinguishes clearly between offline evaluation on the labelled dataset and the live demonstration on captured traffic.

4.1.1 Offline Model Comparison

All four models were evaluated on the same held-out test set of 200,498 flows that the models never saw during training. Their overall results are summarised in Table 4.1. The models reach very similar, high accuracy, but differ substantially in macro F1—the metric that reflects performance on the rare attack classes.

Table 4.1: Model Performance Comparison

Model	Accuracy	Macro F1	Weighted F1	Train time (s)
HistGradientBoosting	0.9803	**0.8627**	0.9822	25.9
XGBoost	**0.9862**	0.8421	0.9846	114.8
MLP	0.9837	0.7373	0.9811	9212.0
Random Forest	0.8991	0.8082	0.9363	54.0

The comparison of accuracy and macro F1 across the models is shown visually in Figure 4.1 and Figure 4.2.

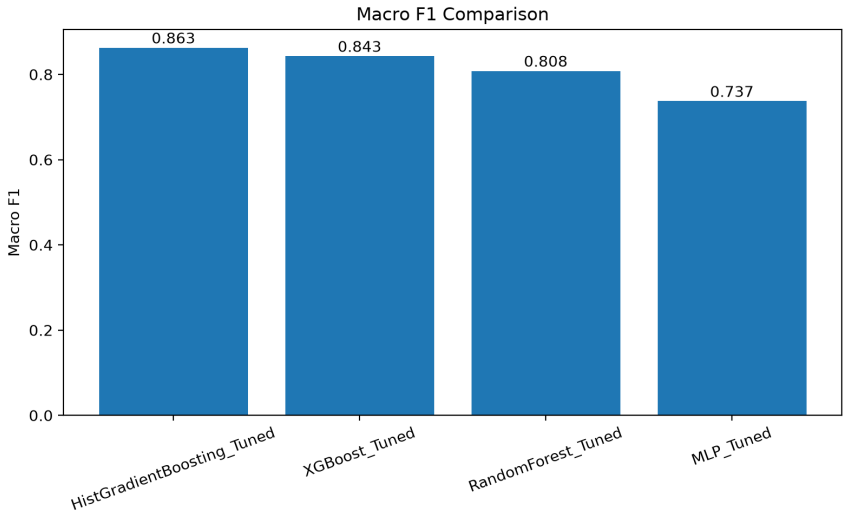

Figure 4.1: Macro F1 Comparison Across Models

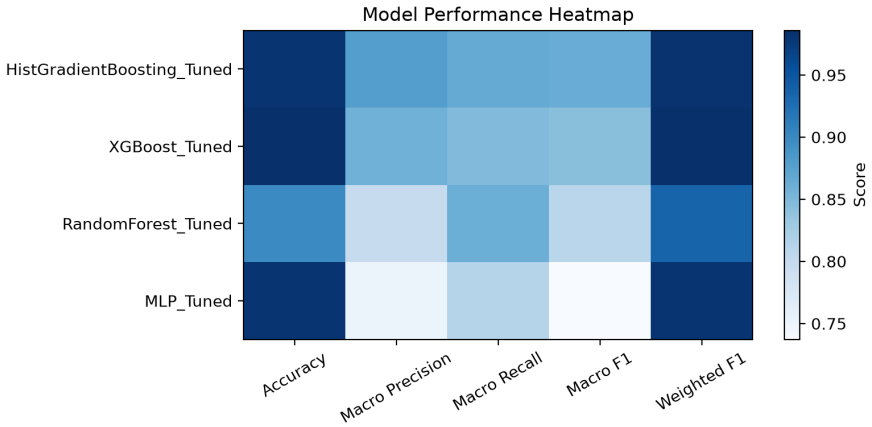

Figure 4.2: Model Performance Heatmap

4.1.2 Best-Performing Model

HistGradientBoosting was selected as the default detection model. Although XGBoost attained the highest raw accuracy (0.9862), HistGradientBoosting achieved the best macro F1-score (0.8627 versus 0.8421) while training in roughly a quarter of the time. Because the dataset is heavily imbalanced, macro F1 is the more meaningful criterion: it gives equal weight to the rare attack classes, which are precisely the classes an intrusion detector must not miss. HistGradientBoosting gives up about half a percentage point of accuracy in exchange for two points of macro F1, which is the better trade-off for an IDS. The MLP, despite competitive accuracy, had by far the lowest macro F1 (0.7373) and the longest training time, confirming that high accuracy alone does not indicate good rare-class detection.

4.1.3 Per-Class and Rare-Class Performance

The per-class results of the selected model on the test set are summarised in Table 4.2. The model classifies the high-volume classes almost perfectly (several attack classes reach an F1 of 1.00), while the hardest classes are the rare and behaviourally subtle ones—*Infilteration* and *SQL Injection*—which is consistent with the wider literature on this dataset.

Table 4.2: Per-Class Results — HistGradientBoosting

Class	Precision	Recall	F1	Support
Benign	0.9924	0.9855	0.9889	170,202
Bot	0.9997	0.9997	0.9997	3,573
Brute Force -Web	0.7037	0.9344	0.8028	122
Brute Force -XSS	0.9565	0.9565	0.9565	46
DDOS attack-HOIC	1.0000	1.0000	1.0000	8,461
DDOS attack-LOIC-UDP	0.9716	0.9884	0.9799	346
DDoS attacks-LOIC-HTTP	0.9992	0.9986	0.9989	7,293
DoS attacks-GoldenEye	0.9906	1.0000	0.9953	525
DoS attacks-Hulk	1.0000	1.0000	1.0000	5,505
DoS attacks-SlowHTTPTest	0.6911	0.5366	0.6041	246
DoS attacks-Slowloris	0.9846	0.9846	0.9846	130
FTP-BruteForce	0.7942	0.8835	0.8365	498
Infilteration	0.2423	0.3753	0.2945	2,049
SQL Injection	0.8571	0.3529	0.5000	17
SSH-Bruteforce	0.9973	1.0000	0.9987	1,485
Macro avg	0.8787	0.8664	0.8627	200,498
Weighted avg	0.9845	0.9803	0.9822	200,498

A per-model comparison on the rare classes is shown in Figure 4.3, and the confusion matrix of the selected model is shown in Figure 4.4.

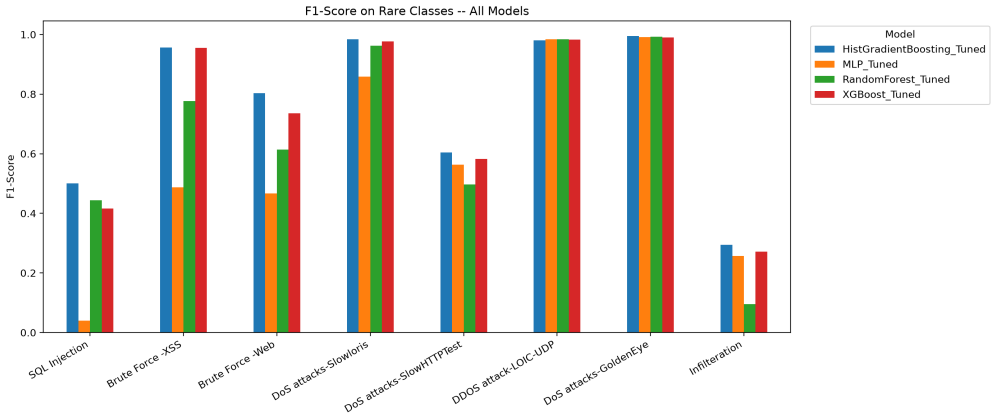

Figure 4.3: Rare Attack Class F1 Comparison

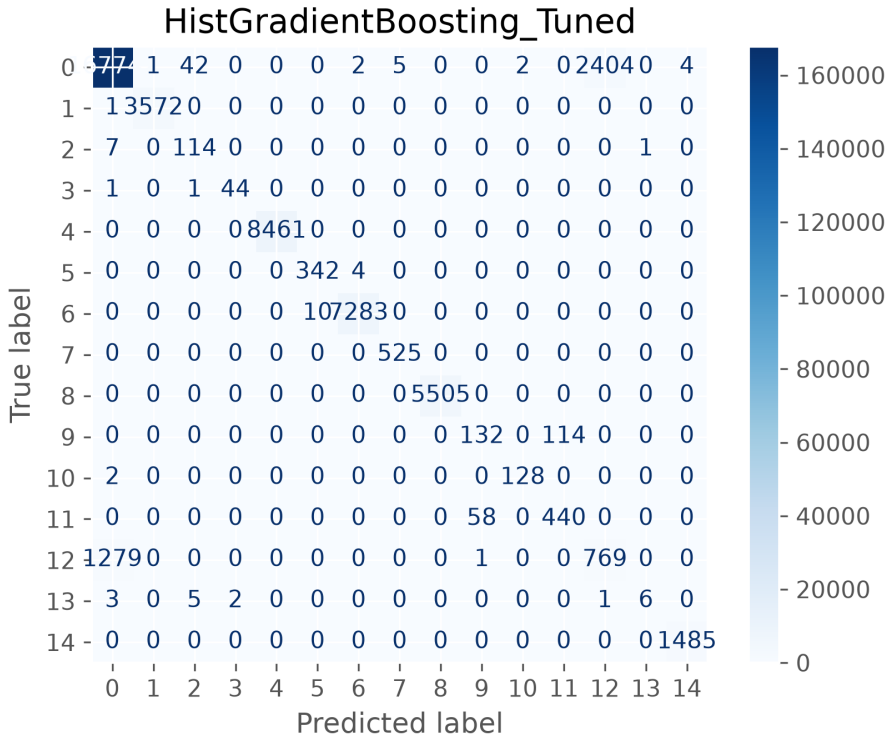

Figure 4.4: Confusion Matrix — HistGradientBoosting

4.1.4 Effect of Training-Data Volume

A supplementary experiment retrained every model at three training-set sizes with frozen hyperparameters and a fixed test set, so that only the amount of data varied. The macro F1 results are shown in Table 4.3 and Figure 4.5. Accuracy barely changes across an eleven-fold increase in data, whereas macro F1 improves markedly—most of all for XGBoost, which gains over eleven points. This confirms that, for this imbalanced problem, additional data helps the rare classes (macro F1) far more than it helps overall accuracy.

Table 4.3: Macro F1 vs Training Set Size

Model	Small (25k)	Medium (100k)	Large (281k)	Gain
HistGradientBoosting	0.8218	0.8466	0.8627	+0.041
XGBoost	0.7316	0.8182	0.8421	+**0.111**
Random Forest	0.7452	0.7892	0.8085	+0.063
MLP	0.7216	0.7521	0.7490	+0.027

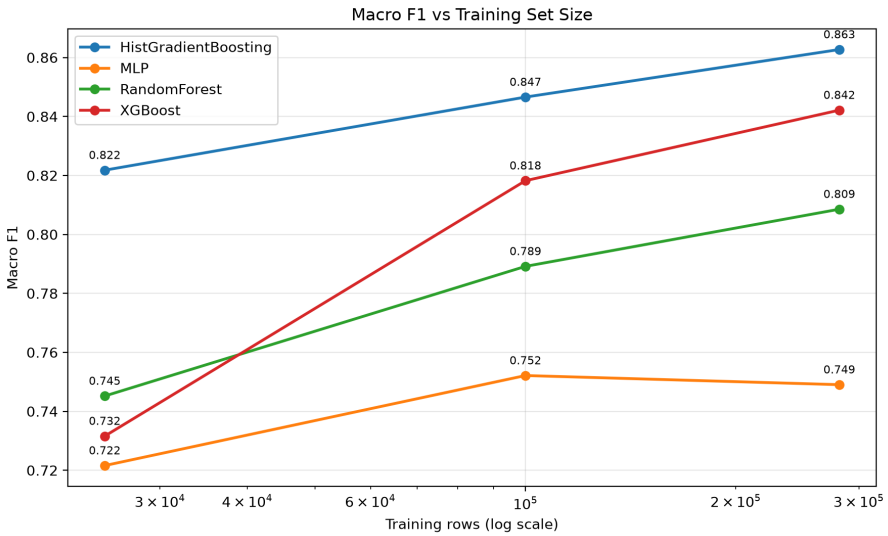

Figure 4.5: Macro F1 vs Training Set Size (Graph)

4.1.5 Web Application and Live Demonstration

The trained model was integrated into the Django web application described in Section 3.2.9, which provides single-flow classification, batch classification with optional evaluation, live monitoring and a history log. The main dashboard, live monitor and batch-evaluation views are shown in Figures 4.6–4.8.

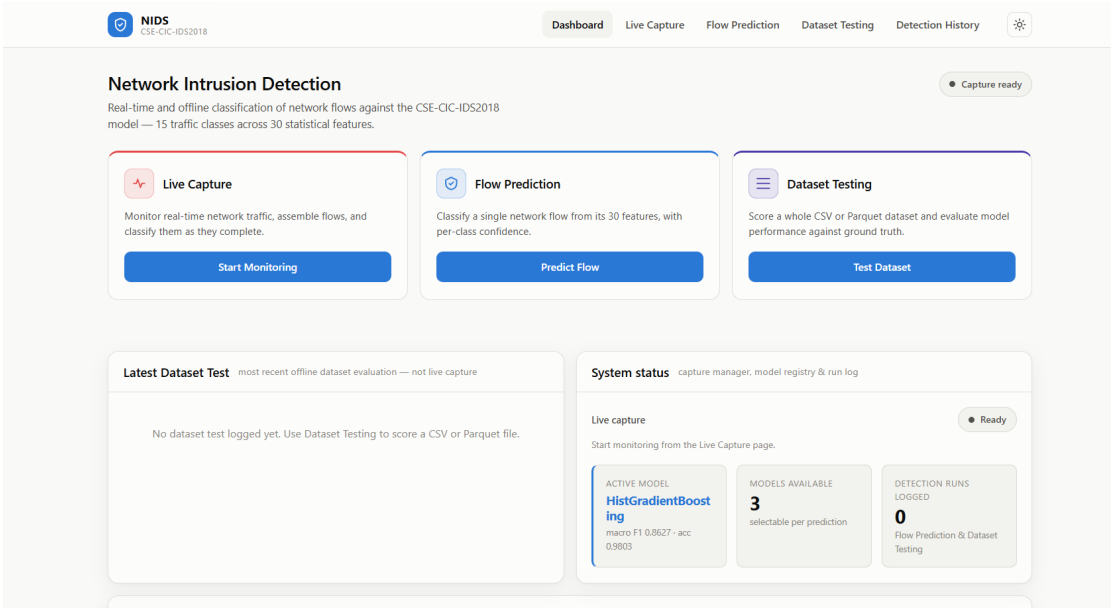

Figure 4.6: NIDS Main Dashboard

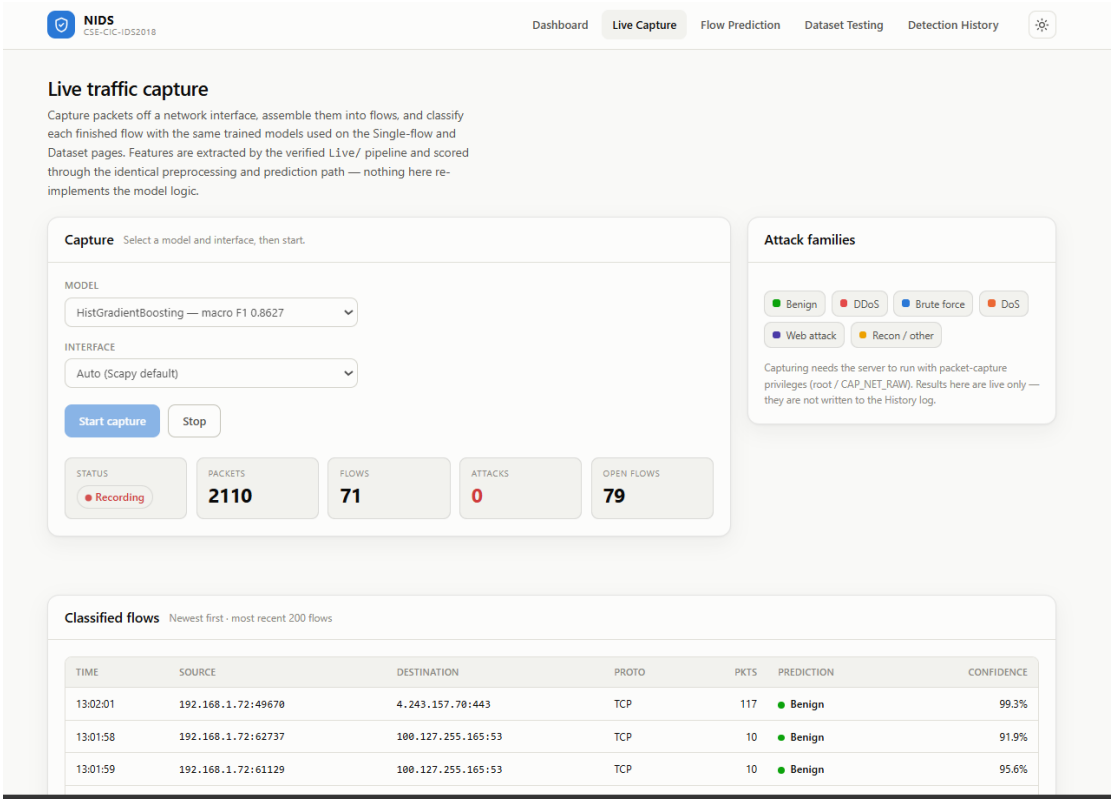

Figure 4.7: Live Traffic Monitoring Page

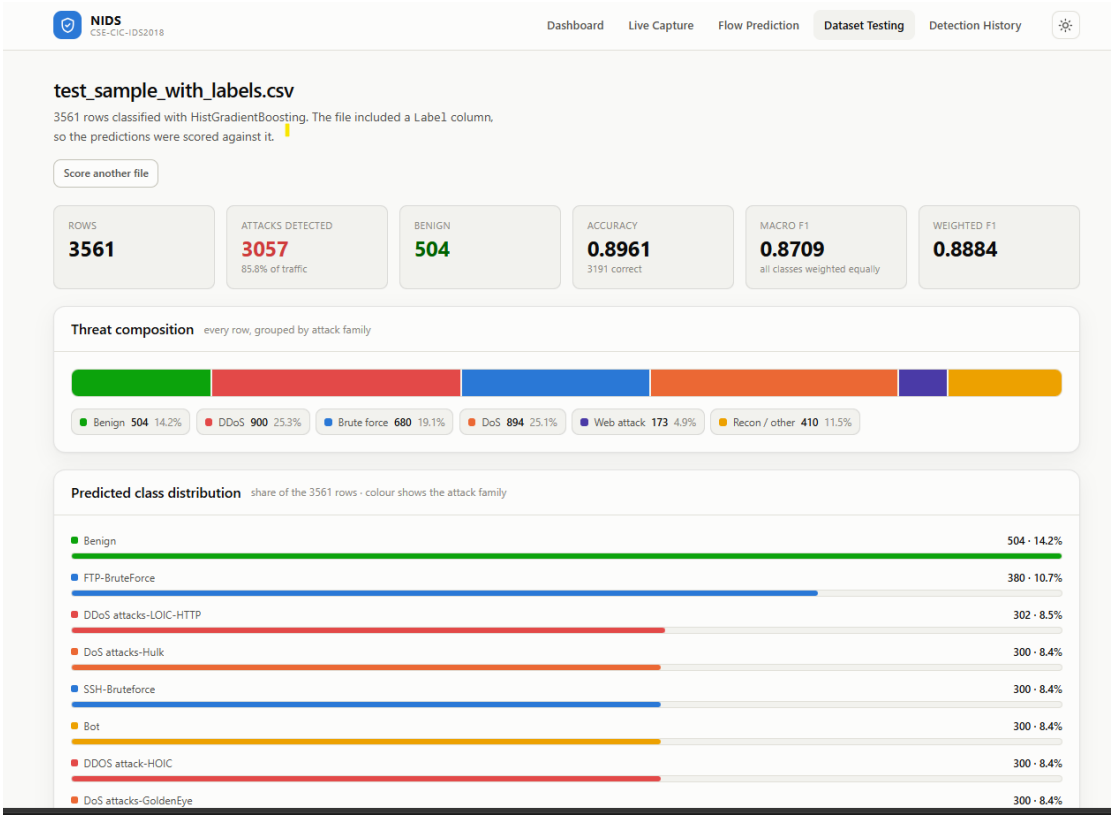

Figure 4.8: Batch Classification and Evaluation

As experimental validation of live detection, a controlled FTP brute-force attack was generated in a local test environment. The resulting network traffic was captured by the NIDS and successfully classified as *FTP-BruteForce*, demonstrating the capability of the system to detect attack traffic in a live environment. This live demonstration is distinct from the offline evaluation above: the offline results quantify detection accuracy on the labelled dataset, whereas the live demonstration confirms that the end-to-end pipeline—capture, flow generation, feature extraction, preprocessing and prediction—functions on real captured traffic.

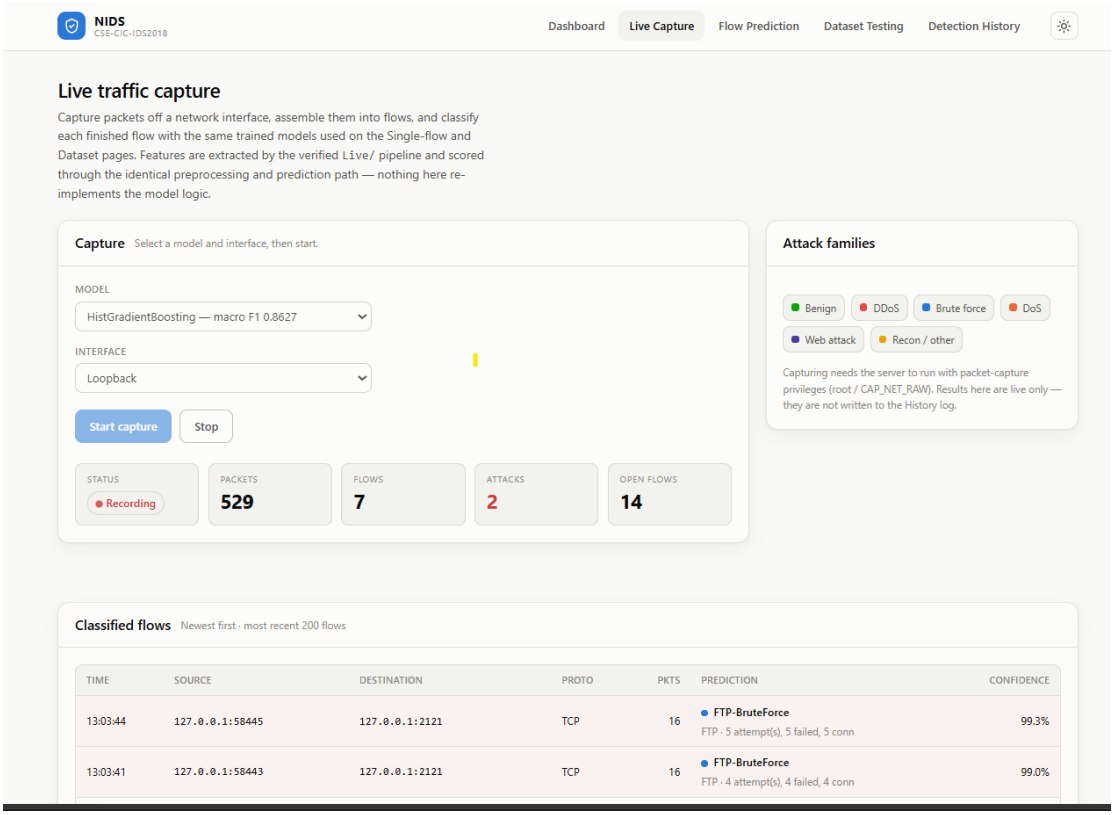

Figure 4.9: Live Attack Detection

4.1.6 Conclusion

This project set out to build a machine-learning and deep learning-based Network Intrusion Detection System and to connect it to live traffic. All of its objectives were met. A large, realistic dataset (CSE-CIC-IDS2018) was cleaned and prepared, reduced to thirty informative flow features by mutual information, and balanced across fifteen traffic classes. Four models were tuned and compared, and HistGradientBoosting was selected as the default detector on the basis of its leading macro F1-score of 0.8627 at 98.03% accuracy. The trained model was integrated into a web application offering single-flow, batch and live detection, and a controlled live-attack demonstration confirmed end-to-end operation on real captured traffic.

Although the models achieve high overall accuracy, detecting rare attack types is still difficult. In particular, Infilteration and SQL Injection are harder for the models to identify correctly. The performance differences between the models, as well as the improvements gained from adding more training data, are most noticeable for these rare attack classes. Evaluating on macro F1 rather than accuracy was therefore essential to an honest assessment. The project demonstrates that machine learning provides an effective and automatable basis for network intrusion detection, while highlighting class

imbalance and rare-attack detection as the key remaining challenges.

4.2 Future Enhancement

Several realistic improvements would extend the system beyond its current laboratory scope. None of these is implemented in the present project; they are directions for future work.

- **Larger and more varied training data:** training on a larger fraction of the full dataset, which the scaling study suggests would further improve rare-class detection.
- **Cross-dataset validation:** evaluating the trained model on other intrusion-detection datasets to measure how well it generalises beyond CSE-CIC-IDS2018.
- **More diverse network environments and attack types:** validating and extending the system on additional network conditions and a wider range of attacks.
- **Improved real-time performance:** optimising the live-capture pipeline for higher-throughput links.
- **Explainable AI:** integrating feature-attribution methods so that each detection can be explained to an analyst.
- **Distributed deployment:** running detection across multiple sensors or network segments.
- **Alerting and response:** adding automated alerts and integration with response tooling when an attack is detected.
- **Continuous learning:** periodically retraining the model on newly observed traffic so that it adapts to evolving threats.
- **Encrypted-traffic analysis:** extending detection to encrypted traffic, where payload inspection is not possible.
- **Better scalability:** hardening the web application and capture pipeline for sustained operational use.

References

[1] I. Sharafaldin, A. Habibi Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pp. 108–116, SciTePress, 2018.

[2] Z. K. Maseer, R. Yusof, N. Bahaman, S. A. Mostafa, and C. F. M. Foozy, "Benchmarking of machine learning for anomaly based intrusion detection systems in the CICIDS2017 dataset," *IEEE Access*, vol. 9, pp. 22351–22370, 2021.

[3] Z. Ahmad, A. S. Khan, C. W. Shiang, J. Abdullah, and F. Ahmad, "Network intrusion detection system: A systematic study of machine learning and deep learning approaches," *Transactions on Emerging Telecommunications Technologies*, vol. 32, no. 1, p. e4150, 2021.

[4] B. A. Tama and S. Lim, "Ensemble learning for intrusion detection systems: A systematic mapping study and cross-benchmark evaluation," *Computer Science Review*, vol. 39, p. 100357, 2021.

[5] R. Vinayakumar, M. Alazab, K. P. Soman, P. Poornachandran, A. Al-Nemrat, and S. Venkatraman, "Deep learning approach for intelligent intrusion detection system," *IEEE Access*, vol. 7, pp. 41525–41550, 2019.

[6] Z. Wang, J. Liu, and L. Sun, "EFS-DNN: An ensemble feature selection-based deep learning approach to network intrusion detection system," *Security and Communication Networks*, vol. 2022, p. 2693948, 2022.

[7] G. Kocher and G. Kumar, "Machine learning and deep learning methods for intrusion detection systems: Recent developments and challenges," *Soft Computing*, vol. 25, no. 15, pp. 9731–9763, 2021.

[8] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pp. 2623–2631, ACM, 2019.

[9] J. L. Leevy and T. M. Khoshgoftaar, "A survey and analysis of intrusion detection models based on CSE-CIC-IDS2018 big data," *Journal of Big Data*, vol. 7, no. 1, p. 104, 2020.

[10] L. Liu, G. Engelen, T. Lynar, D. Essam, and W. Joosen, "Error prevalence in NIDS datasets: A case study on CIC-IDS-2017 and CSE-CIC-IDS-2018," in *2022 IEEE Conference on Communications and Network Security (CNS)*, pp. 254–262, IEEE, 2022.

[11] G. Karataş, Ö. Demir, and O. K. Sahingoz, "Increasing the performance of machine learning-based IDSs on an imbalanced and up-to-date dataset," *IEEE Access*, vol. 8, pp. 32150–32162, 2020.

[12] L. Xu, M. Skoularidou, A. Cuesta-Infante, and K. Veeramachaneni, "Modeling tabular data using conditional GAN," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

Bibliography

- Canadian Institute for Cybersecurity, University of New Brunswick. *CSE-CIC-IDS2018 Dataset.* https://www.unb.ca/cic/datasets/ids-2018.html
- Canadian Institute for Cybersecurity. *CICFlowMeter — Network Traffic Flow Feature Extractor.* https://www.unb.ca/cic/research/applications.html
- Django Software Foundation. *Django Documentation.* https://docs.djangoproject.com/
- Scapy Community. *Scapy: Packet Manipulation Library.* https://scapy.net/
- scikit-learn Developers. *scikit-learn User Guide.* https://scikit-learn.org/stable/user_guide.html
- XGBoost Developers. *XGBoost Documentation.* https://xgboost.readthedocs.io/
- Optuna Developers. *Optuna: A Hyperparameter Optimization Framework.* https://optuna.org/
- The pandas Development Team. *pandas Documentation.* https://pandas.pydata.org/docs/

Screenshots

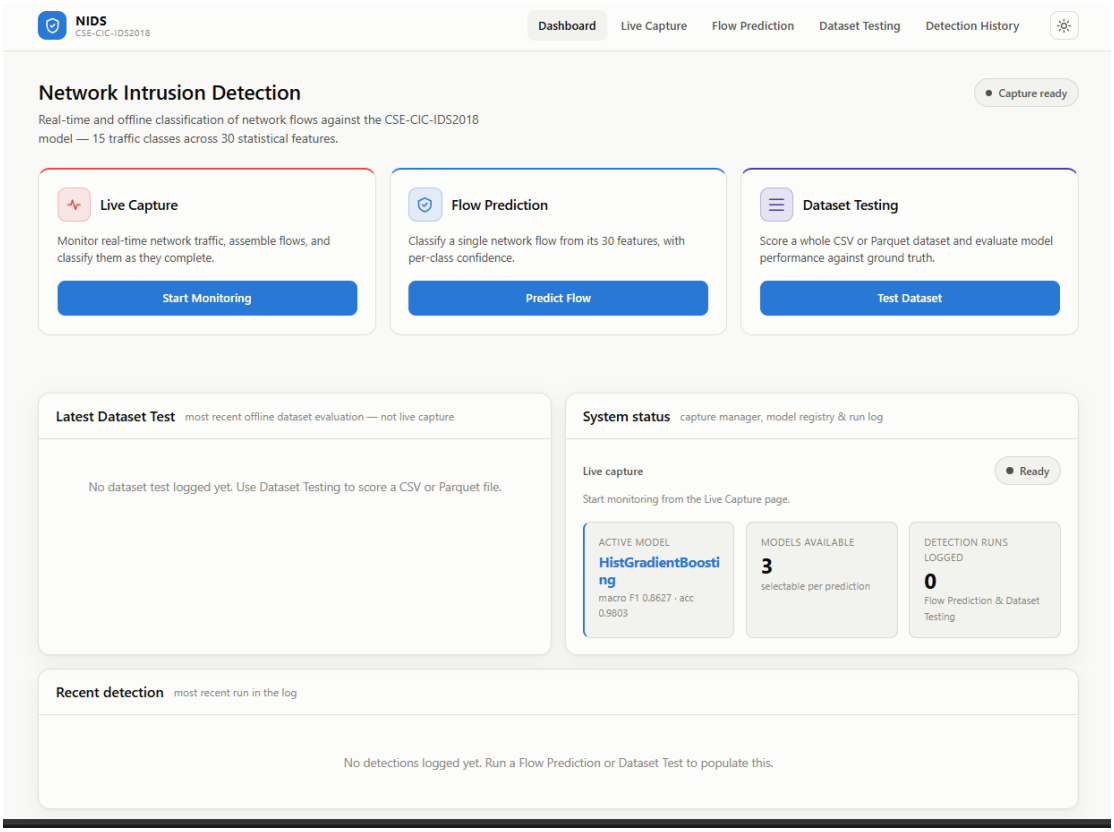

Figure 4.10: NIDS Main Dashboard

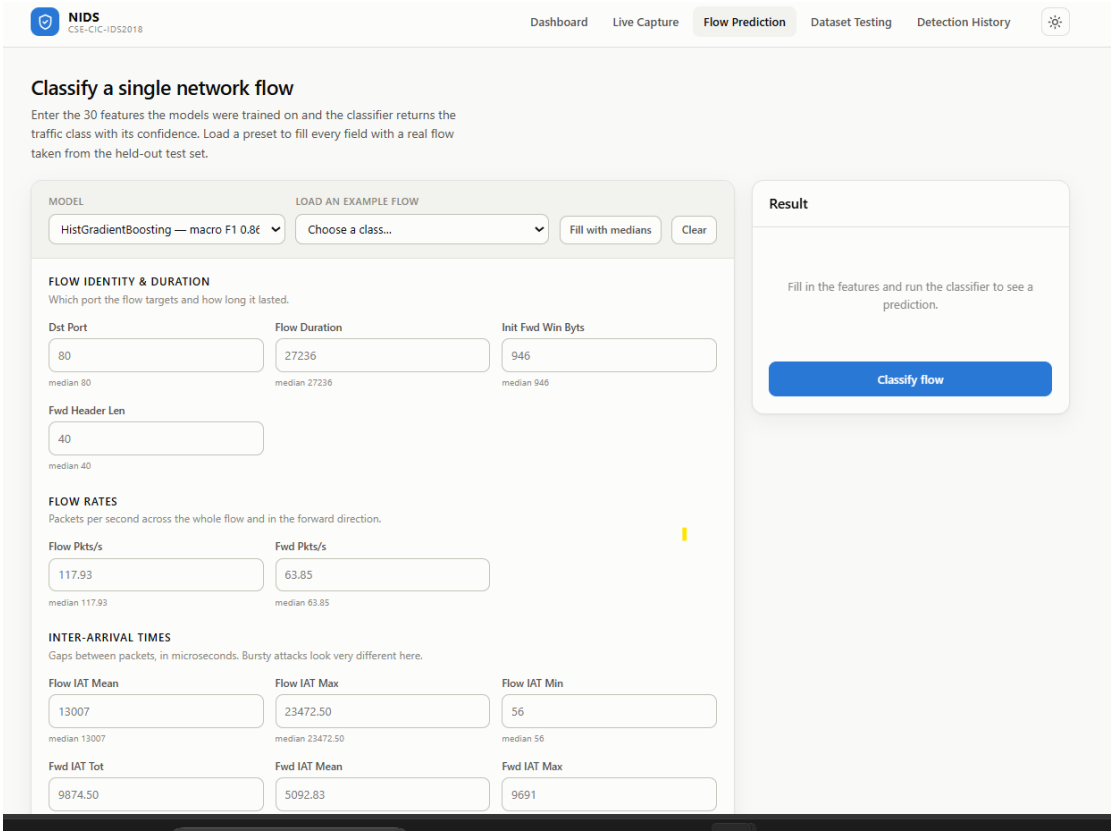

Figure 4.11: Flow Prediction Page

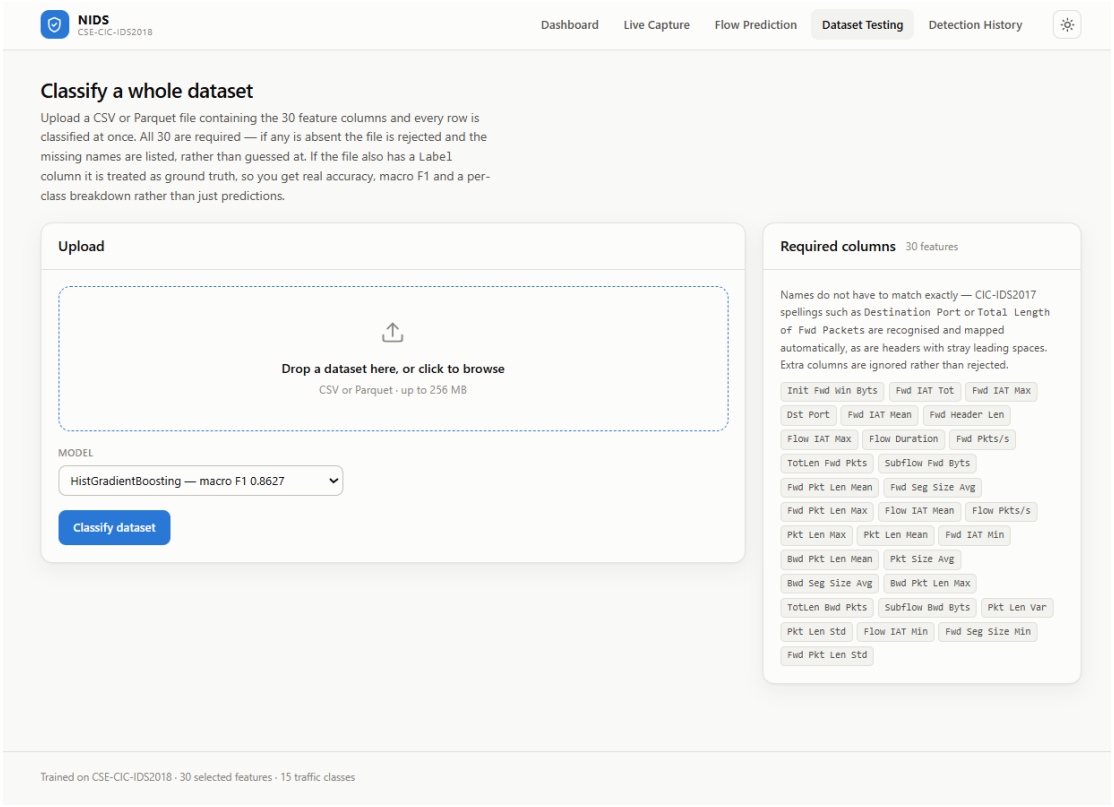

Figure 4.12: Dataset Testing Page

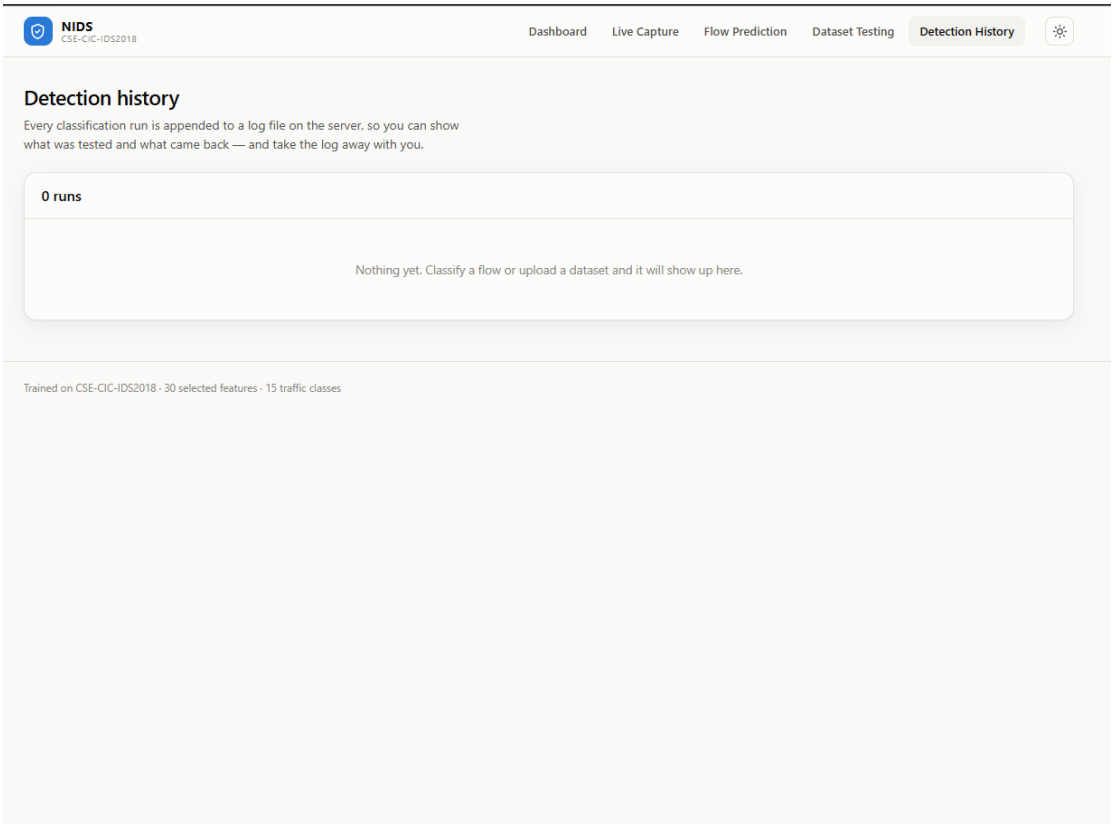

Figure 4.13: Detection History Page